TECHNICAL DEVELOPER REFERENCE

3D GUI System
Complete Backend Developer Guide

Project	Open RAN Digital Twin Platform
System	3D GUI + 2D Grafana Monitoring + Simulation Controller
Backend	FastAPI (controller.py port 8001) + Python 2D GUI (port 8000)
Frontend	Three.js 3D Scene + Vite Dev Server (port 3001)
Database	InfluxDB 1.8 (time-series KPMs) + SQLite (decision log)
Author	Omar Farouk
Date	May 06, 2026
Chapters	17 chapters | Complete system reference

Abstract

This guide is the complete technical reference for the 3D GUI backend system of the Open RAN Digital Twin Platform. It covers every component from the FastAPI orchestration controller (port 8001) that launches and manages the simulation, through the InfluxDB time-series database and SQLite decision log, to the Three.js 3D frontend and the Grafana 2D dashboard backend. Every API route is documented with its request body, response format, and step-by-step internal logic. Every file in the codebase is explained. The guide is intended for a developer who wants to understand, modify, or extend the system.

TABLE OF CONTENTS

3D GUI System — Complete Backend Developer Guide

Total: 20 chapters covering every file, every API route, every data flow, and every development scenario.

CHAPTER 1

System Overview

1.1 What This GUI Is

The 3D GUI is a live command center for the O-RAN simulation platform. While the simulation runs, the GUI shows you exactly what is happening inside the network: which cells are overloaded, where each User Equipment (UE) is physically located, when handovers fire, and whether the GRU neural network is making correct handover decisions. You can also start and stop every component of the simulation from a single web page without touching the terminal.

Think of it as an air traffic control radar. The radar does not fly the planes — the planes fly themselves (the ns-3 simulation and FlexRIC RIC handle all the radio logic). But the radar gives the controller (you) a real-time picture of everything in the airspace, and it lets the controller issue commands. The 3D canvas is the radar screen; the cell towers are the airports; the UE spheres are the aircraft; and the glowing arcs that appear during handovers are the transfer beams switching a plane from one approach path to another.

> **Key Point:** The GUI never modifies the simulation logic itself. It only reads data from InfluxDB, starts/stops processes via subprocess, and writes results to files and SQLite. If the GUI crashes, the simulation continues running.

1.2 The Two GUI Layers

The system intentionally has two separate GUI layers, each optimised for a different purpose.

Layer 1 — 2D Grafana Dashboard (port 3000)

Grafana is the traditional monitoring layer. It displays time-series charts of every KPM (Key Performance Metric) that ns-3 reports: PRB usage per cell, SINR per UE, throughput, latency, error rates. Grafana reads directly from InfluxDB using InfluxQL queries. It is excellent for long-horizon trend analysis, and its dashboards are pre-provisioned from JSON files so they appear automatically with no manual setup.

Layer 2 — 3D Three.js Scene (port 3001)

The 3D scene provides spatial, real-time situational awareness. Cell towers are rendered as geometrically accurate mast structures with parabolic satellite dishes. UEs appear as small glowing spheres positioned at their actual ns-3 (x, y) coordinates scaled to the Three.js world. When a handover fires, an expanding ring of three concentric circles blooms at the destination cell, and a white luminous arc connects the source tower to the destination tower along a Bezier curve. The 3D view is updated every 1.5 seconds by polling the 2D backend through the Vite proxy.

> ***Note:*** *The 3D scene and 2D Grafana do NOT share a frontend — they are completely separate web applications. The 3D scene (5g-gui-v2/) fetches data from the 2D backend (GUI/main.py) via its Vite proxy, so the 2D backend must be running for the 3D scene to display live data.*

1.3 Full Component Table

The table below lists all nine components of the system with their port, implementation language, and role.

Component	Port	Language	Role
controller.py	8001	Python / FastAPI	Host-level orchestrator: starts/stops all processes, saves simulation results, exposes /ctrl/* API

Component	Port	Language	Role
sim_data_pusher.py	—	Python	Reads ns-3 CSV output files every 3 seconds and writes measurements to InfluxDB
GUI/main.py (2D backend)	8000	Python / FastAPI	Queries InfluxDB, builds structured simulation state, serves /refresh-data and Grafana data
InfluxDB 1.8	8086	Go (Docker)	Time-series database storing all UE and cell KPMs produced by the simulation
Grafana	3000	Go (Docker)	Pre-provisioned dashboards reading InfluxDB; 2D chart monitoring layer
Vite dev server / 3D frontend	3001	JavaScript / Three.js	3D scene rendering, SINR sparklines, system control panel, handover animation
FlexRIC nearRT-RIC	36421 (SCTP)	C	Near-real-time RAN Intelligent Controller — receives E2 reports, sends RC control messages
ns-3 simulation	—	C++	Network simulator running the gru_scenario; generates all CSV KPM files and handover.csv
GRU Python inference service	5000	Python / Flask	Receives feature vectors from xapp_handover_gru, returns predicted best target cell

1.4 Why Each Component Exists

A common question is why there are so many separate processes instead of one monolithic program. The answer is that each component is specialised and can fail or be restarted independently.

- **controller.py is separate from the 2D backend** because it needs to run as a host process with direct access to the filesystem and subprocess API. The 2D backend runs inside a Docker container, which deliberately isolates it from the host.
- **sim_data_pusher.py is separate from ns-3** because ns-3 is a C++ binary and cannot natively speak InfluxDB's line protocol. The pusher is a Python bridge that reads the CSV files ns-3 writes and translates them into InfluxDB writes.
- **InfluxDB is separate from SQLite** because they store fundamentally different data. InfluxDB is a time-series database optimised for high-frequency numeric metrics — exactly what KPMs are. SQLite is a relational database suited for structured, queryable records like the decision log where you need to filter by sim label, UE ID, or correctness flag.
- **The 3D frontend polls the 2D backend rather than InfluxDB directly** because InfluxDB 1.8 does not have a JavaScript client library and does not support CORS. The 2D backend does the InfluxQL queries and serves the result as clean JSON.

CHAPTER 2

Directory Structure — Every File Explained

This chapter walks through every file that matters in the system. Understanding where things live is essential before you can debug or extend the code.

2.1 5g-gui-v2/ — The 3D Frontend and Controller

This directory is the root of the 3D GUI application. It contains the FastAPI controller backend, the generate_plots standalone script, the Vite frontend project, and all compiled assets. Run 'npm run dev' from this directory to start the Vite development server.

5g-gui-v2/controller.py

The FastAPI backend that runs on port 8001. This is the brain of the entire system: it holds references to every running subprocess, exposes the /ctrl/* REST API used by the 3D frontend, orchestrates the 10-step launch-all sequence, and saves simulation results to disk and SQLite when the simulation finishes. If this file is missing, the system control panel in the 3D GUI will show all components as OFFLINE and the launch-all button will do nothing.

5g-gui-v2/generate_plots.py

A standalone post-simulation analysis script. It reads handover.csv from a sim directory, detects ping-pong events, generates four matplotlib plots, writes decision_log.csv, decision_summary.json, and summary.txt. It can be run manually after a simulation even if the controller did not run. If this file is missing, post-simulation plots will not be generated by kill_sim.sh.

5g-gui-v2/index.html

The HTML entry point served by the Vite dev server. It loads the Three.js application via a module script tag pointing to src/main.js. It also defines the full DOM skeleton of the page: the canvas element, all panel divs, cell cards container, handover log, SINR strip, system control form, and scenario selector. If this file is missing, the browser will see a blank page.

5g-gui-v2/vite.config.js

Vite build configuration. It sets the dev server port to 3001 and defines four proxy rules so the browser can reach backend services without CORS errors. Without this file, running 'npm run dev' will fail, and even if it somehow started, all fetch() calls to /api, /ctrl, /xapp-start, and /xapp-stop would be blocked by the browser's same-origin policy.

5g-gui-v2/src/main.js

The JavaScript entry point and application coordinator. It initialises the Three.js scene by calling initScene(), sets up the two polling intervals (pollCtrlStatus every 3 seconds, pollBackend every 1.5 seconds), handles all DOM interactions (launch-all button, stop-all button, individual component start/stop buttons), and detects handovers by comparing UE serving cell IDs between successive /refresh-data responses. If this file is missing or has syntax errors, the entire 3D application will fail to load.

5g-gui-v2/src/scene.js

The Three.js rendering module. It exports initScene(), setUEPositions(), triggerHandover(), and flashUELabel(). It builds the entire 3D world: the ground plane, all cell towers (each composed of a steel mast, parabolic dish, strut arms, feed horn, and glow sphere), UE spheres with connecting lines to their serving tower, and the handover animation system (expanding concentric rings and Bezier arc). If this file is missing, the canvas will be blank and import errors will break main.js.

5g-gui-v2/src/config.js

The single source of truth for all static configuration: the ALL_CELLS array defining the 8 cells (1 LTE macro + 7 mmWave) with their coordinates, initial load, and color; the CONFIG object with RF parameters, load thresholds, xApp rules, and scenario definitions; and the BACKEND_URL constant ('/api'). If this file is missing, both main.js and scene.js will crash at import time.

5g-gui-v2/src/ui.js

DOM manipulation helpers for the status panel, cell cards, handover log, and alert system. This file is imported by main.js and provides functions that update specific HTML elements without touching the Three.js scene. If it is missing, the cell card updates and handover log will fail.

5g-gui-v2/src/agent.js

The AI agent module that integrates a Gemini language model. It provides a floating chat interface where the user can ask natural-language questions about the current simulation state. The agent receives the current NET state object as context. This file is optional for core simulation monitoring — if missing, the agent chat panel simply will not work, but all other GUI features remain functional.

5g-gui-v2/src/style.css

All CSS for the 3D GUI application: the dark HUD theme, cell card styling, handover log entries, SINR sparkline strip, system control panel, and responsive layout. If this file is missing, the page will render unstyled with broken layout.

2.2 GUI/ — The 2D Backend (Docker)

This directory lives at yousef_fathy/ns-O-RAN-flexric/mmwave-LENA-oran/GUI/ and is the Docker Compose project for the 2D monitoring stack.

GUI/main.py

The FastAPI application entry point for the 2D backend, running on port 8000 inside the Docker container. It mounts static files, includes the influx_data_router from data_controller.py, and runs a stop_simulation() call on startup to clear stale state. If this file is missing, the Docker container will fail to start, the /refresh-data endpoint will be unreachable, and the 3D GUI will fall back to demo mode.

GUI/docker-compose.yml

Defines three Docker services: influxdb (influxdb:1.8-alpine on port 8086), gui (the Python 2D backend on port 8000, built from the local Dockerfile), and grafana (grafana/grafana:8.0.2 on port 3000 with pre-provisioned dashboards). This file is what controller.py calls when you press 'Start Docker' or 'Launch All'. If this file is missing, 'docker compose up' will fail and nothing will start.

GUI/configuration.env

Environment variables for Grafana and InfluxDB: admin credentials (GF_SECURITY_ADMIN_USER=admin, INFLUXDB_ADMIN_USER=admin), Grafana plugin list, and InfluxDB database name (influx). These values are injected into the containers at startup. If this file is missing, Docker will start but with wrong credentials and the Grafana datasource will fail to authenticate.

GUI/src/http/data_controller.py

The FastAPI router that implements the /refresh-data endpoint. This is what the 3D frontend polls every 1.5 seconds. It calls SimulationManager.refresh_simulation() to query InfluxDB, then returns a JSON object containing ues (list of UE dataclass dicts), cells (list of cell dataclass dicts), max_x_max_y, simulation_status, energy metrics, and SINR/PRB maps. If this file has errors, all live data in the 3D scene will stop updating.

GUI/src/simulation_objects/simulation.py

The Simulation class that queries InfluxDB. Its constructor creates an InfluxDBClient using the four environment variables (host, port, username, password). The get_simulation_data() method issues a SELECT * FROM LIMIT 1 query for every UE field and every cell field. The get_charts_max_axis_value() method reads gnbs_x_0 and gnbs_y_0 measurements to determine the simulation world size. If this file is missing, the Simulation class cannot be instantiated.

GUI/src/simulation_objects/simulation_manager.py

A singleton manager that holds the current Simulation instance. refresh_simulation() creates a new Simulation object (triggering fresh InfluxDB queries), and get_simulation() returns the cached instance. This design means every /refresh-data call gets a fresh snapshot of the database.

GUI/src/simulation_objects/cell.py and ue.py

Python dataclasses that define the schema for Cell and Ue objects. These are what get serialised to JSON by the /refresh-data endpoint. Cell has fields like cell_id, dlPrbUsage_percentage, MeanActiveUEsDownlink, x_position, y_position. Ue has fields like ue_id, x_position, y_position, MMWave_Cell, L3servingSINR_dB, and all KPM measurements. If these files are missing, data_controller.py cannot import them.

2.3 Root Files

sim_decisions.db

SQLite database at /home/omar_farouk/open-ran-clean/sim_decisions.db. Created automatically by controller.py on first save. Stores the decisions table with one row per executed handover across all simulation runs. If this file is deleted, historical decision data is lost permanently — it cannot be recovered from InfluxDB.

gru.sh

Shell script that provides a terminal-based alternative to the GUI for launching a full simulation run. It starts Docker, FlexRIC, the GRU Python service, the ns-3 simulation, the data pusher, and the xApp in sequence with appropriate wait steps. If this file is out of date after a fix, the terminal workflow will use stale paths or parameters.

kill_sim.sh

Shell script that kills all simulation processes cleanly and then runs generate_plots.py on the latest sim directory to produce post-run plots. Step 5 of kill_sim.sh calls: python3 /home/omar_farouk/open-ran-clean/5g-gui-v2/generate_plots.py. If this file is missing or has a wrong path, killing the simulation will not generate plots.

2.4 3D_GUI_Sim_Results/ — Simulation Output Archive

Every completed simulation run produces a numbered subdirectory here following the naming pattern sim###_YYYYMMDD_HHMMSS_tag (e.g. sim010_20260505_210259_gru_scenario). Each directory contains the following files.

File	Description
handover.csv	Raw handover events logged by ns-3: time_sec, ue_id, from_cell, to_cell, event, executed_ok
decision_log.csv	Enriched version of handover.csv: adds uuid, sim label, is_correct (ping-pong flag)
decision_summary.json	Aggregate statistics: total handovers, ping-pong count and rate, GRU accuracy
summary.txt	Human-readable text version of decision_summary.json
plots/decision_quality.png	Scatter plot: green dots for correct handovers, red X marks for ping-pong events
plots/handovers_over_time.png	Line plot of cumulative handover count vs simulation time
plots/ho_per_ue.png	Bar chart showing how many handovers each UE performed
plots/ho_activity.png	Histogram of handovers in 5-second windows across the simulation duration
flexric.log / simulation.log / etc.	Copies of the runtime log files from /tmp/ at the time the simulation ended

CHAPTER 3

controller.py — Complete API Reference

3.1 Setup and Global State

controller.py starts by creating a FastAPI application with the title 'Farouk GUI Controller' and adding CORSMiddleware with allow_origins=['*']. This allows the Vite dev server running on port 3001 to call the controller on port 8001 without the browser blocking the request due to same-origin policy. In production, you would restrict origins to the known frontend hostname.

Two critical module-level globals hold shared state across all requests. The _procs dict maps string keys (e.g. 'flexric', 'simulation', 'pusher') to subprocess.Popen objects. The _last_result dict holds the return value of the most recent save_sim_results() call, which the /ctrl/last-result endpoint exposes to the frontend.

The LOG dict maps component names to log file paths under /tmp/. FlexRIC always writes to /tmp/flexric.log because the nearRT-RIC binary is hardcoded to write there — there is no command-line flag to redirect it. All other log paths (farouk_ns3.log, farouk_pusher.log, farouk_xapp.log, farouk_gru.log) are custom paths chosen to avoid polluting the simulation working directory.

> ***Note:*** *The SQLite database is at /home/omar_farouk/open-ran-clean/sim_decisions.db. The RESULTS_DIR is /home/omar_farouk/open-ran-clean/3D_GUI_Sim_Results. The HANDOVER_CSV source is /home/omar_farouk/handover.csv (where ns-3 writes live). These paths are constants at the top of the file — edit them if you move the project.*

3.2 Process Management Internals

Three helper functions implement the process lifecycle: _popen(), _kill(), and _alive().

- **_popen(key, cmd, cwd, log_key, shell, env)** kills any existing process under the same key, then spawns a new subprocess.Popen with stdout and stderr both redirected to the log file, the given working directory, preexec_fn=os.setsid to create a new process group, and the optional custom environment. Storing the Popen object under _procs[key] allows later calls to _kill() and _alive() to find it.
- **_kill(key)** pops the process from _procs and sends SIGTERM to its entire process group via os.killpg(os.getpgid(pid), SIGTERM). This is important for shell=True processes where the Popen object is the shell, not the actual binary — killing only the shell would leave the child ns-3 binary running as an orphan.
- **_alive(key)** checks _procs[key].poll() for non-None (None means still running). For the 'simulation' key there is a special fallback: it runs pgrep -f 'build/scratch/ns3.42-' because ns-3 is launched via shell=True and the Popen object points to the shell wrapper, which exits immediately after spawning ns-3. The pgrep ensures _alive('simulation') correctly returns True while ns-3 is actually running.

3.3 API Routes — Read Operations

GET /ctrl/status

Returns a JSON object with five boolean fields: docker, flexric, simulation, pusher, xapp. Each is computed by calling the appropriate _alive() variant. The 'docker' flag calls _docker_running() which runs 'docker compose ps --services --filter status=running' in the GUI_DIR and checks for non-empty output. The 'xapp' flag calls _xapp_any_alive() which returns True if either the 'xapp_gru' key is alive or the xapp_rc_handover_ctrl binary is running according to ps aux.

GET /ctrl/scenarios

Scans the NS3_DIR/scratch directory for .cc files and returns their stems (filename without extension) as a sorted JSON array. This is what populates the scenario dropdown in the 3D GUI's system control panel. If the scratch directory is empty or missing, the response will be an empty list.

GET /ctrl/logs/{component}

Returns the last N lines (default 40, configurable via ?lines= query parameter) of the log file for the named component. The component name is looked up in the LOG dict. If the file does not exist (e.g. the component has never been started), the endpoint returns {'lines': []}. This drives the log viewer panel in the frontend.

GET /ctrl/decisions

Queries the SQLite decisions table and returns a list of decision records. Accepts an optional ?sim= filter (e.g. ?sim=sim005) to restrict results to a single run, and an optional ?limit= parameter (default 500). Returns {'decisions': [...]} where each element has uuid, sim, time_sec, ue_id, from_cell, to_cell, is_correct, saved_at. Returns {'decisions': [], 'message': 'No decisions recorded yet'} if the database file does not exist.

GET /ctrl/last-result

Returns the _last_result global dict which is populated by save_sim_results() at the end of each simulation. Contains folder path, sim label, ping-pong stats, and accuracy percentage. Returns {'status': 'no results yet'} before any simulation has completed in the current controller session.

3.4 API Routes — Docker Control

POST /ctrl/docker/start

Runs 'docker compose up -d influxdb gui' in GUI_DIR using subprocess.run() (not Popen — this is synchronous and waits for the command to finish). Returns the last 400 characters of stdout+stderr so the frontend can show any error message. Note that only influxdb and gui are started here, not grafana — grafana is optional and can be started separately.

POST /ctrl/docker/stop

Runs 'docker compose down' in GUI_DIR. This stops and removes all containers defined in docker-compose.yml. InfluxDB data is preserved in the named volume influxdb_data because 'down' without --volumes does not delete volumes.

3.5 API Routes — FlexRIC Control

POST /ctrl/flexric/start

Calls _popen('flexric', [FLEXRIC_BIN], cwd=NS3_DIR, log_key='flexric'). The working directory is NS3_DIR rather than the FlexRIC build directory because FlexRIC needs to find its configuration and E2AP

shared libraries relative to that path. Returns 'already_running' if _alive('flexric') is True.

POST /ctrl/flexric/stop

Calls _kill('flexric') then also runs pkill -f nearRT-RIC as a belt-and-suspenders kill. This is necessary because sometimes FlexRIC spawns child processes that survive SIGTERM to the process group.

3.6 API Routes — Simulation Control

POST /ctrl/simulation/start (request body: SimParams)

Accepts a JSON body matching the SimParams Pydantic model: scenario (string, default 'gru_scenario'), n_ues (int, default 20), n_mmwave (int, default 7), sim_time (int, default 60), e2_term_ip (string, default '127.0.0.1'). Before starting ns-3, it clears stale InfluxDB data by running a DELETE FROM /..*/ query via docker exec. Then it constructs the ns3 run command with all flags and calls _popen with shell=True.

```
./ns3 run "scratch/gru_scenario.cc --e2TermIp=127.0.0.1 --hoSinrDifference=3
--indicationPeriodicity=0.05 --simTime=60 --KPM_E2functionID=2 --RC_E2functionID=3
--N_MmWaveEnbNodes=7 --N_Ues=20"
```

POST /ctrl/simulation/stop

Calls _kill('simulation') then pkill -9 -f 'gru_scenario|ns3.42' to ensure the actual C++ binary is terminated. The -9 (SIGKILL) is used here because ns-3 sometimes ignores SIGTERM while still in its main simulation loop.

3.7 API Routes — Pusher and xApp Control

POST /ctrl/pusher/start and POST /ctrl/pusher/stop

Start spawns 'python3 sim_data_pusher.py' in NS3_DIR using _popen. Stop kills it and also runs pkill -f sim_data_pusher. The pusher needs to run in NS3_DIR because it uses os.listdir('.') to discover CSV files produced by the simulation.

POST /ctrl/xapp/start

This endpoint implements a two-step xApp start for the non-GRU scenario path. First it ensures xApp_trigger.py is running (the trigger server listens on port 38868). Then it sends an HTTP POST with body 'start' to http://localhost:38868/. The trigger server receives this and launches xapp_rc_handover_ctrl. Returns 'started' on success or 'error' with the exception detail if the HTTP call fails.

POST /ctrl/xapp/stop

Sends an HTTP POST with body 'stop' to http://localhost:38868/ (the trigger server's stop signal), then also pkill -f xapp_rc_handover_ctrl as a safety net.

3.8 POST /ctrl/launch-all — The Big One

This is the most important endpoint. A POST to /ctrl/launch-all starts a FastAPI BackgroundTask that runs the _launch_all_task async coroutine. The HTTP response returns immediately with {'status': 'launching'} — the actual work happens in the background. The task follows a strict 10-step sequence.

0. **Kill everything stale.** All known processes are killed by name (pkill -9 for xApp binaries, data pusher, GRU service, nearRT-RIC, ns3.42) and all _procs entries are cleared. /tmp/flexric.log is truncated so E2 connection counting starts fresh. Then the coroutine sleeps 3 seconds to let OS file handles close.

1. **Start Docker (InfluxDB + 2D GUI backend).** Runs docker compose up -d influxdb gui. Then polls http://localhost:8086/ping in a loop (max 30 seconds) until InfluxDB responds. Once InfluxDB is ready, sends a DROP DATABASE influx followed by CREATE DATABASE influx. This wipes all stale field schemas — critical because InfluxDB 1.8 caches field types and a mismatch between runs causes write errors.
2. **Start FlexRIC nearRT-RIC.** Spawns the nearRT-RIC binary with -c flexric.conf. Then polls 'ss -anp | grep :36421' for up to 60 seconds until the SCTP E2 port 36421 is open. After the port opens, waits an additional 2 seconds as margin. If FlexRIC is not alive after this, the task returns early.
3. **GRU Python inference service (GRU scenario only).** Spawns 'python3 gru_xapp.py' with the GRU_PORT environment variable set to 5000. Sleeps 3 seconds to let Flask start. Then clears the three runtime CSVs (handover.csv, lstm_features.csv, kpm_handover_features.csv) to empty headers — necessary because ns-3 appends to these files and stale data from the previous run would corrupt ping-pong detection.
4. **Start ns-3 simulation.** Same command as /ctrl/simulation/start. After spawning, the coroutine waits for E2 connections by counting ESTABLISHED SCTP sessions on port 36421 with 'ss -Snp | grep :36421 | grep -c ESTAB'. It waits until the count reaches n_mmwave (default 7), polling every 3 seconds for up to 3 minutes. This ensures the xApp is not started before all gNBs have connected to FlexRIC.
5. **Start data pusher.** Spawns sim_data_pusher.py, waits 3 seconds.
6. **Start xApp.** For GRU scenario: spawns xapp_handover_gru directly with the LSTM_SERVICE_URL environment variable pointing to localhost:5000. For other scenarios: ensures the trigger server is running then sends a 'start' HTTP POST.
7. **Wait for simulation to finish.** Polls _alive('simulation') every 5 seconds in a while loop. The coroutine does nothing else while waiting — it simply blocks until ns-3 exits naturally at simTime seconds.
8. **Save results.** Calls save_sim_results(tag=params.scenario) and stores the return value in _last_result.

> ***Important: launch-all is an async background task. If the controller process is killed while launch-all is running (e.g. via Ctrl+C), save_sim_results() will NOT be called and the handover.csv from that run will not be archived. Always use POST /ctrl/stop-all to gracefully end a run, which saves results, rather than killing the controller.***

3.9 POST /ctrl/stop-all

Sends 'stop' to the trigger server, kills all _procs entries, and runs pkill for all known process names. Note that stop-all does NOT call save_sim_results() — it is a pure emergency shutdown. Results are only saved automatically when the simulation finishes naturally (launch-all task step 8) or by calling generate_plots.py manually.

3.10 save_sim_results() Function Breakdown

This function is the bridge between a completed simulation run and the permanent record. It performs six steps.

- **Step 1 — Assign sim number and create directory.** _next_sim_number() counts directories in RESULTS_DIR that start with 'sim' and have three digits (simXXX pattern) and returns count+1. The directory name is sim{N:03d}_{timestamp}_{tag}.

- **Step 2 — Copy data files.** handover.csv and lstm_features.csv are copied from their runtime locations (/home/omar_farouk/) to the new directory. Component log files from /tmp/ are also copied with their component name as the filename.
- **Step 3 — Compute ping-pong stats.** _calc_pingpong() reads the copied handover.csv, filters for executed_ok==1 rows, groups them by UE, and checks each consecutive pair: if A→B followed by B→A within 5 seconds, that is one ping-pong. Returns total, pingpong, rate_pct.
- **Step 4 — Build decision log.** _build_decision_log() does the same ping-pong detection but returns a list of enriched dicts: each executed handover gets a UUID (uuid.uuid4()), the sim label, and an is_correct boolean (False if it is the first leg of a ping-pong pair). The second leg is NOT marked as incorrect — only the event that caused the bounce back is penalised.
- **Step 5 — Write to SQLite.** _write_decisions_to_db() uses INSERT OR IGNORE so re-running save_sim_results on the same CSV will not create duplicate rows. The UUID is the primary key ensuring idempotency.
- **Step 6 — Generate plots and summary files.** _generate_plots() creates the plots/subdirectory and saves decision_quality.png and handovers_over_time.png using matplotlib with Agg backend (no display needed). decision_summary.json and summary.txt are written with aggregate statistics.

CHAPTER 4

generate_plots.py — Standalone Post-Simulation Analysis

4.1 Purpose and Two Modes

generate_plots.py is a standalone script that can be run at any time after a simulation, even if the controller never ran. It reads the raw handover.csv from a simulation directory, applies the ping-pong detection algorithm, generates four matplotlib plots, and writes the analysis files. There are two ways to invoke it.

```
python3 /home/omar_farouk/open-ran-clean/5g-gui-v2/generate_plots.py
```

Auto-detect mode: scans RESULTS_DIR for all simXXX directories, sorts them by modification time, and picks the most recently modified one. Prints 'Auto-detected: ' before running.

```
python3 /home/omar_farouk/open-ran-clean/5g-gui-v2/generate_plots.py
/path/to/sim010_20260505_210259_gru_scenario
```

Explicit path mode: uses the provided directory directly. The trailing slash is stripped by rstrip('/') before processing.

4.2 Internal Flow — 8 Steps

1. **Locate handover.csv.** The script builds the path os.path.join(sim_dir, 'handover.csv') and calls sys.exit(1) if it does not exist. This is the only required input file.
2. **Read and filter rows.** csv.DictReader reads all rows. Only rows where executed_ok=='1' are kept. These represent handovers that were actually executed by the xApp, not just reported.
3. **Sort by time.** Rows are sorted by float(r['time_sec']) to ensure correct temporal ordering for ping-pong detection.
4. **Detect ping-pong events.** See section 4.3 for the full algorithm. The result is a set of row indices (pp_indices) that are marked as ping-pong.
5. **Build decisions list.** Each kept row becomes a dict with uuid, sim label, time_sec, ue_id, from_cell, to_cell, and is_correct = (index not in pp_indices).
6. **Compute aggregate metrics.** total=len(decisions), pp_count=len(pp_indices), correct=total-pp_count, accuracy=correct/total*100, pp_rate=pp_count/total*100.
7. **Write output files.** decision_log.csv, decision_summary.json, summary.txt are written to the sim directory.
8. **Generate four plots.** Each plot is saved as a PNG at 150 DPI in the plots/ subdirectory.

4.3 Ping-Pong Detection Algorithm

The ping-pong detection algorithm identifies harmful handover oscillations where a UE is handed from cell A to cell B and then immediately back from cell B to cell A within a 5-second window. This indicates the xApp made an unnecessarily aggressive handover decision.

The algorithm groups executed handovers by UE ID. Grouping by UE is critical: without it, an interleaved handover from a different UE could break a valid A→B, B→A pair detection for the UE you are examining. For each UE's sorted list of handovers, the algorithm checks each consecutive pair (i-1, i):

- Condition 1: rows[i-1]['to_cell'] == rows[i]['from_cell'] — the previous handover destination equals the current handover source (the UE is bouncing back from the same cell)
- Condition 2: rows[i-1]['from_cell'] == rows[i]['to_cell'] — the previous handover source equals the current handover destination (the UE is returning to where it came from)
- Condition 3: float(rows[i]['time_sec']) - float(rows[i-1]['time_sec']) <= 5.0 — the round trip happened within 5 seconds

When all three conditions are true, the index of rows[i-1] (the first leg, the outgoing A→B handover) is added to pp_indices. The return trip (B→A) is NOT marked. This convention means a ping-pong counts as one event that costs one decision's worth of accuracy penalty.

> **Key Point:** The 5-second window is not arbitrary — it matches the TTT (Time-To-Trigger) and hysteresis window used in the xApp configuration. A handover that bounces within the TTT window is almost certainly due to SINR oscillation at a cell edge, not genuine mobility.

4.4 The Four Plots

Plot 1: decision_quality.png

A scatter plot with simulation time on the X-axis and a binary Y-axis (0=Ping-pong, 1=Correct). Green dots represent correct handovers. Red X marks (marker='x', linewidths=2, alpha=0.9) represent ping-pong events. The title includes both the PP rate and the accuracy percentage so the most important numbers are visible without reading the JSON file.

Plot 2: handovers_over_time.png

A cumulative handover count line plot. The X-axis is simulation time in seconds; the Y-axis is the running total of handovers. A steep slope indicates a burst of handovers; a flat region means no handovers occurred. The title shows total count.

Plot 3: ho_per_ue.png

A bar chart showing total handover count per UE ID. Uses collections.Counter to aggregate. UE IDs are sorted numerically on the X-axis. This reveals whether load is distributed evenly across UEs or concentrated on a few mobility-heavy users.

Plot 4: ho_activity.png

A histogram of handovers bucketed into 5-second windows using numpy.histogram with bin_edges = np.arange(0, 61, 5). The X-axis is the start of each 5-second window; the Y-axis is handover count in that window. Coral-colored bars with black edges. This reveals whether the xApp is uniformly active throughout the simulation or shows bursty behavior in specific periods.

4.5 Integration with kill_sim.sh

kill_sim.sh step 5 calls generate_plots.py in auto-detect mode. This means every time you kill the simulation via the kill script, plots are automatically generated for that run. The auto-detect mode finds the most recently

modified sim directory, which will be the one that was just populated by save_sim_results() — or by the handover.csv file if save_sim_results() was not triggered.

4.6 Output Files Table

Output File	Location	Format
decision_log.csv	sim_dir/	CSV: uuid, sim, time_sec, ue_id, from_cell, to_cell, is_correct
decision_summary.json	sim_dir/	JSON: sim, tag, timestamp, total_handovers, pingpong_events, pingpong_rate_pct, correct_decisions, total_decisions, accuracy_pct
summary.txt	sim_dir/	Human-readable plain text with same statistics
decision_quality.png	sim_dir/plots/	Scatter plot PNG at 150 DPI, 12x4 inches
handovers_over_time.png	sim_dir/plots/	Line plot PNG at 150 DPI, 12x4 inches
ho_per_ue.png	sim_dir/plots/	Bar chart PNG at 150 DPI, 12x4 inches
ho_activity.png	sim_dir/plots/	Histogram PNG at 150 DPI, 12x4 inches

CHAPTER 5

sim_data_pusher.py — InfluxDB Bridge

5.1 Role and Location

sim_data_pusher.py lives at yousef_fathy/ns-O-RAN-flexric/mmwave-LENA-oran/sim_data_pusher.py and must be run from that same directory. It is the bridge between the ns-3 simulation CSV output files and InfluxDB. ns-3 writes KPM data to CSV files as it runs; the pusher reads those files every 3 seconds and writes the new measurements to InfluxDB; the 2D backend then queries InfluxDB to build the simulation state for the frontend.

Without the pusher running, the 3D GUI will show cells and UEs frozen at their initial positions and no real KPM data will appear. The simulation will still run correctly — ns-3 and FlexRIC are unaffected — but the GUI will be blind.

5.2 InfluxDB Connection

The pusher connects to InfluxDB using the influxdb Python client library. The connection parameters are hardcoded in the main() function: host='localhost', port=8086, user='root', password='root', db_name='influx'. Note that these are not the same credentials as in configuration.env (which uses 'admin'/'admin') — this is a known quirk. InfluxDB 1.8 with no authentication mode accepts any credentials, so both work. client.create_database(db_name) is called on each push to ensure the database exists even if InfluxDB was just restarted.

5.3 Which CSV Files Are Read

The main() function defines two categories of files. Core files are fixed: ue_position.txt, gnbs.txt, and enbs.txt. These always exist (or should exist) once ns-3 has started writing output. Additional files are discovered dynamically by scanning the current directory for files matching cu-cp-cell-*.txt, cu-up-cell-*.txt, or du-cell-*.txt. The number of these files depends on n_mmwave (7 mmWave cells means 7 du-cell files, etc.).

- **ue_position.txt** — UE positions and serving cell IDs, updated as UEs move
- **gnbs.txt** — gNB (mmWave base station) positions
- **enbs.txt** — eNB (LTE base station) positions
- **cu-cp-cell-N.txt** — CU-CP KPMs per cell: SINR, RRC metrics
- **cu-up-cell-N.txt** — CU-UP KPMs per cell: throughput, PDCP volume
- **du-cell-N.txt** — DU KPMs per cell: PRB usage, active UEs, latency

5.4 Two Push Functions

push_count_to_influx() — for core files

For ue_position.txt, gnbs.txt, and enbs.txt, the pusher calls push_count_to_influx() which reads the file without clearing it, finds the most recent timestamp row, counts the number of unique IDs at that timestamp, and writes

a single measurement '{filename}_count' with field value=count. This is how the 2D backend knows how many UEs and gNBs are active.

push_data_to_influx() — for all files

This is the main data push function. It calls read_and_clear_csv() which reads the file and immediately rewrites it with just the header row — this prevents accumulating duplicate data on the next push cycle. It then iterates over every data row, applies the measurement naming logic described below, and builds a list of InfluxDB point dicts that it writes in a single client.write_points() batch call.

5.5 Measurement Naming Patterns

The InfluxDB measurement name encodes both the data source and the entity ID. The naming convention varies by file type.

UE KPM measurements (from cu-cp / cu-up / du cell files, ue_fields set)

When a column header is in the ue_fields set, the measurement name is ue_{ue_id}_{column_name}. The ue_id is taken from fields[1] (the second column) which contains the IMSI or UE index.

Cell KPM measurements (cell_fields set only)

When a column is in cell_fields but not in ue_fields, the measurement name is {filename}_{column_name}. For example, a dlprbusage value from du-cell-3.txt becomes measurement 'du-cell-3_dlprbusage'.

Shared fields (in both sets)

When a column is in both ue_fields and cell_fields (e.g. dlprbusage, rru.prbuseddl), two separate InfluxDB points are written: one as ue_{id}_... and one as {filename}_...

Core file fields (ue_position.txt, gnbs.txt)

These use the core files path which calls push_data_to_influx with file_path in core_files. For these files, every column (except timestamp) gets its own measurement named {filename}_{column_name}_{record_id}. This is how UE positions become measurements named ue_position_x_1, ue_position_y_1, ue_position_cell_1, and gNB positions become gnbs_x_0, gnbs_y_0 (the 0 index is the world bounds sentinel).

5.6 All Measurement Names Reference Table

Measurement Name Pattern	Example	Meaning
ue_{id}_l3 serving sinr	ue_5_l3 serving sinr	UE 5 serving cell SINR in dB
ue_{id}_l3 neigh sinr {1-8}	ue_3_l3 neigh sinr 2	UE 3 neighbor cell 2 SINR
ue_{id}_l3 serving id(m_cellid)	ue_1_l3 serving id(m_cellid)	UE 1 serving cell ID
ue_{id}_drb.pdcpsdubitratedl.ueid(pdcpthroughput)	ue_7_drb.pdcpsdubitratedl...	UE 7 DL PDCP throughput
ue_{id}_rru.prbuseddl	ue_2_rru.prbuseddl	UE 2 DL PRB usage count
ue_{id}_drb.pdcpsdudelaydl.ueid(pdcp latency)	ue_4_drb.pdcpsdudelaydl...	UE 4 PDCP latency

Measurement Name Pattern	Example	Meaning
du-cell-{N}_dlprbusage	du-cell-3_dlprbusage	Cell 3 DL PRB usage percentage
du-cell-{N}_drb.meanactiveuedl	du-cell-1_drb.meanactiveuedl	Cell 1 mean active UEs DL
du-cell-{N}_drb.pdcpsdudelaydl (cellaveragelatency)	du-cell-2_drb...	Cell 2 average DL latency
ue_position_x_{id}	ue_position_x_12	UE 12 X coordinate in ns-3 meters
ue_position_y_{id}	ue_position_y_12	UE 12 Y coordinate in ns-3 meters
ue_position_cell_{id}	ue_position_cell_3	UE 3 currently serving cell index
gnbs_x_{id}	gnbs_x_0	gNB 0 X coordinate (index 0 = world max X)
gnbs_y_{id}	gnbs_y_0	gNB 0 Y coordinate (index 0 = world max Y)
ue_position_count	—	Total number of active UEs at latest timestamp
gnbs_count	—	Total number of active gNBs

CHAPTER 6

InfluxDB — Time-Series Database

6.1 What Is a Time-Series Database?

A relational database like SQLite stores data in tables with rows and columns. You can JOIN tables, GROUP BY arbitrary columns, and update individual rows. This flexibility comes at the cost of write throughput — every write must maintain ACID guarantees across the entire schema. A time-series database (TSDB) makes a different tradeoff: every data point is associated with a timestamp, and writes are append-only. This makes TSDBs extremely fast for exactly the workload the simulation produces: thousands of numeric KPM samples per second, each timestamped, never updated.

InfluxDB 1.8 organises data into measurements (analogous to SQL tables), tags (indexed string metadata, used for filtering), fields (the actual numeric values, not indexed), and time (an implicit column in every row). A measurement name like 'ue_5_l3 serving sinr' is its own measurement — there is no single UE table; each UE-field combination is a separate time series. This denormalised design is intentional: it allows the 2D backend to query a single measurement and get back only the field it needs without scanning all UE data.

6.2 All Measurement Types Stored

The measurements stored in InfluxDB are grouped into four categories.

UE Position Measurements

ue_position_x_{id}, ue_position_y_{id}, ue_position_cell_{id}, and ue_position_type_{id} for each UE. These come from ue_position.txt and are used by the 2D backend to set the x_position, y_position, and MMWave_Cell fields in the Ue dataclass.

UE KPM Measurements

ue_{id}_l3 serving sinr, ue_{id}_l3 neigh sinr 1 through 8, ue_{id}_l3 neigh id 1 through 8, ue_{id}_drb.pdcpsdubitratedl.ueid(pdcpthroughput), ue_{id}_rru.prbuseddl, ue_{id}_drb.pdcpsdudelaydl.ueid(pdcp latency), ue_{id}_drb.buffersize.qos.ueid, ue_{id}_tb.errtotalnbrdl.1.ueid. These come from the cu-cp, cu-up, and du cell CSV files and populate the KPM fields of the Ue dataclass.

Cell KPM Measurements

du-cell-{N}_dlprbusage (the primary cell load indicator), du-cell-{N}_drb.meanactiveuedl, du-cell-{N}_drb.pdcpsdudelaydl (cellaveragelatency), cu-cp-cell-{N}_sinr, cu-up-cell-{N}_m_pdcpbytesdl. These populate the Cell dataclass fields used for cell load bars and Grafana cell charts.

gNB Position Measurements

gnbs_x_{id}, gnbs_y_{id} for each gNB. gnbs_x_0 and gnbs_y_0 are the world dimension sentinels used by get_charts_max_axis_value() in simulation.py to determine the ns-3 coordinate space bounds. Without these, the 2D backend defaults to 6000x6000 meters.

6.3 How simulation.py Queries InfluxDB

The Simulation class uses a single helper method get_last_value_from_measurement() which executes: SELECT * FROM {measurement} LIMIT 1. InfluxDB returns time-series data in reverse-chronological order by default when using LIMIT, so LIMIT 1 returns the most recent data point. The method extracts the 'value' field from the result. This SELECT * FROM ... LIMIT 1 pattern is called for every UE field and every cell field on every /refresh-data request — which means dozens of InfluxDB queries per request. This is acceptable for a development/demo system but would need batching in a production deployment.

6.4 Docker Setup

InfluxDB runs as the 'influxdb' service in docker-compose.yml. Key configuration:

- **Image: influxdb:1.8-alpine** — Alpine-based for small image size. Version 1.8 uses the original InfluxQL query language and the /query HTTP endpoint. Version 2.x uses Flux query language and a completely different API — the Python influxdb client library (not influxdb-client) is compatible only with 1.x.
- **Port: 127.0.0.1:8086:8086** — Bound to localhost only (127.0.0.1) for security. External machines cannot reach InfluxDB directly. The data pusher runs on the host and connects to localhost:8086.
- **Volume: influxdb_data** — Named Docker volume at /var/lib/influxdb inside the container. Data persists across container restarts. The volume is NOT deleted by 'docker compose down' (only by 'docker compose down --volumes').
- **Startup command** — The influxdb service uses a custom command: 'influxd & sleep 10 && influx -database influx -execute "delete from /\w*/"; tail -f /dev/null'. This starts the daemon, waits 10 seconds, then clears all measurements. The tail keeps the container alive.

6.5 Why InfluxDB Is Dropped and Recreated on Each launch-all

InfluxDB 1.8 caches the data type of each field. If in one run a field contained a numeric float, but in the next run the CSV exports it as a string (e.g. 'N/A' or empty), InfluxDB will reject the write with a type conflict error. The only reliable fix is to drop and recreate the database before each run. The launch-all task does this with two curl commands after InfluxDB is ready:

```
curl -s -X POST "http://localhost:8086/query" --data-urlencode "q=DROP DATABASE
influx"
curl -s -X POST "http://localhost:8086/query" --data-urlencode "q=CREATE DATABASE
influx"
```

6.6 Credentials

InfluxDB 1.8 in the default configuration does not enforce authentication unless INFLUXDB_HTTP_AUTH_ENABLED is set to true. In this project it is not set, so any credentials work. The configuration.env sets INFLUXDB_ADMIN_USER=admin and INFLUXDB_ADMIN_PASSWORD=admin. The 2D backend uses these from environment variables (INFLUXDB_USERNAME, INFLUXDB_PASSWORD). The data pusher uses 'root'/'root' — both work because auth is not enforced.

6.7 Manual Inspection Commands

```
docker exec -it $(docker ps -q -f name=influxdb) influx
```

```
> USE influx
> SHOW MEASUREMENTS
> SELECT * FROM "ue_5_l3 serving sinr" LIMIT 5
> SELECT * FROM "du-cell-3_dlprbusage" LIMIT 5
> DROP SERIES FROM /.*/ -- clear all data without dropping DB
```

CHAPTER 7

SQLite — The Decisions Database

7.1 Why SQLite Instead of InfluxDB for Decisions

Handover decisions are structurally different from KPM measurements. A KPM is a floating-point number sampled 20 times per second per UE — purely numeric, high-frequency, and you query it by time range. A handover decision is a discrete event with multiple semantic attributes: which UE, from which cell, to which cell, was it correct, what was the simulation it came from. You need to query decisions by sim label (give me all decisions from sim005), by correctness flag (give me all ping-pong events), and join across attributes. This is exactly what relational SQL is designed for.

SQLite requires zero infrastructure — no server process, no Docker container, no network port. The database is a single file at /home/omar_farouk/open-ran-clean/sim_decisions.db. Backup is as simple as copying that file.

7.2 Schema

The decisions table is created by _write_decisions_to_db() with CREATE TABLE IF NOT EXISTS:

```
CREATE TABLE IF NOT EXISTS decisions ( uuid TEXT PRIMARY KEY, sim TEXT, time_sec REAL, ue_id INTEGER, from_cell INTEGER, to_cell INTEGER, is_correct INTEGER, saved_at TEXT )
```

7.3 Column Reference

Column	Type	Example	Explanation
uuid	TEXT PK	a3f2-...	Random UUID assigned at save time. Primary key ensures INSERT OR IGNORE is idempotent — re-saving the same run will not create duplicate rows.
sim	TEXT	sim007	Simulation label derived from the result directory name (first 6 chars). Allows filtering all decisions from a specific run.
time_sec	REAL	23.450	Simulation clock time in seconds when the handover was executed. Matches the time_sec column in handover.csv.
ue_id	INTEGER	12	UE index (1-based) that performed the handover.
from_cell	INTEGER	3	Cell index the UE was served by before the handover.
to_cell	INTEGER	7	Cell index the UE moved to after the handover.
is_correct	INTEGER	1	1 if the handover is not a ping-pong event; 0 if it is the first leg of a bounce-back. Stored as integer because SQLite has no boolean type.

Column	Type	Example	Explanation
saved_at	TEXT	2026-05-05T21:02:59	Host wall-clock datetime when save_sim_results() ran. ISO format string from datetime.now().isoformat().

7.4 How Data Enters the Database

Data enters through _write_decisions_to_db() which is called by save_sim_results() after _build_decision_log() produces the decisions list. The insert uses executemany() with INSERT OR IGNORE — if the uuid already exists (meaning this run was already saved), the insert is silently skipped. This allows safely re-running save_sim_results() on the same handover.csv without creating duplicates. The commit() is called once after all rows are inserted, making the write a single atomic transaction.

7.5 GET /ctrl/decisions API

The /ctrl/decisions endpoint returns decision records from SQLite. It supports two query parameters: ?sim=sim005 to filter by simulation label, and ?limit=500 (default) to cap the number of rows returned. The response is {'decisions': [list of dicts]}. Each dict has all eight columns. The frontend uses this to show the decision history panel and to compute per-simulation accuracy statistics.

```
curl http://localhost:8001/ctrl/decisions?sim=sim007&limit;=100
curl http://localhost:8001/ctrl/decisions
```

7.6 Manual Inspection Commands

```
sqlite3 /home/omar_farouk/open-ran-clean/sim_decisions.db
sqlite> .tables
sqlite> SELECT sim, COUNT(*), SUM(CASE WHEN is_correct=0 THEN 1 ELSE 0 END) FROM
decisions GROUP BY sim;
sqlite> SELECT * FROM decisions WHERE sim='sim007' ORDER BY time_sec;
sqlite> SELECT * FROM decisions WHERE is_correct=0 ORDER BY saved_at DESC LIMIT 20;
sqlite> DELETE FROM decisions WHERE sim='sim001'; -- remove a specific run
```

CHAPTER 8

Frontend Architecture — Three.js + Vite

8.1 main.js — Application Coordinator

main.js is the single JavaScript entry point. When the page loads, it immediately calls initScene() from scene.js to render all 8 cell towers with the full 20-UE layout, even before the backend responds. This ensures the user sees a functional 3D scene instantly rather than a blank canvas.

Boot Sequence

The boot sequence (at the bottom of the file, executed on module load) runs as follows: CONFIG.CELLS is set to ALL_CELLS.slice() (all 8 cells); NET.cells is built with initial loads from config.js; buildCards() populates the cell card HTML; setSimStatus('STOPPED') initialises the status dot; loadScenarios() fetches the scenario list from /ctrl/scenarios; pollCtrlStatus() is called immediately and then every 3 seconds; initScene() renders the 3D world; pollBackend() is called immediately and then every 1.5 seconds.

Polling Loop 1 — pollCtrlStatus (every 3 seconds)

Fetches GET /ctrl/status with a 3-second abort timeout. On success, calls _setCompStatus() for each of the five components (docker, flexric, simulation, pusher, xapp). _setCompStatus() updates the dot color, state text, and card CSS class in the system control panel.

Polling Loop 2 — pollBackend (every 1.5 seconds)

Fetches GET /api/refresh-data (proxied to localhost:8000/refresh-data) with a 3-second abort timeout. On success: extracts ns-3 coordinate bounds (max_x, max_y) and builds the ns3ToScene() coordinate transform function; checks if the number of cells has changed and rebuilds the scene if so; updates cell load values; if simulation is running, processes each UE's serving cell to detect handovers.

Handover Detection

The prevServingCells dict maps UE index to serving cell ID from the previous poll. On each poll, for every UE where the current serving cell differs from the previous one (and both are nonzero), main.js calls triggerHandover(fromCell.x, fromCell.z, toCell.x, toCell.z) in scene.js, flashUELabel(ueIndex) in scene.js, and logHandover() to add an entry to the handover log panel.

Demo Mode

When pollBackend() throws (backend unreachable), NET.simMode is set to true. A setInterval running every 2 seconds applies a random walk to each cell's load value (drift bounded to [-0.25, +0.25] applied at 1.2% per tick). This keeps the 3D scene animated when developing without a running backend.

8.2 scene.js — Three.js Rendering Module

Scene Setup

The initScene() function creates a WebGLRenderer, adds an EffectComposer with UnrealBloomPass for the glow effect on cell towers and UE spheres, sets up perspective camera at radius 70 units, adds a dark ground

plane, directional and ambient lights, and starts the render loop. The bloom pass threshold is tuned so only the brightest materials (emissive glow spheres at the tower tops) produce halos.

Cell Tower Geometry

Each LTE macro tower is built from: a tall steel cylinder mast (height ~52 units), three diagonal tube struts at 120-degree intervals near the top, and a parabolic dish antenna. Each mmWave small cell tower is shorter (~21 units) with the same parabolic dish. The buildParaDish() function constructs a realistic parabolic reflector using a parametric surface with depth = R*0.38 and focal length F = R^2/(4*depth). The dish is made from a dense triangular mesh (MeshStandardMaterial with metalness=0.72, roughness=0.10) oriented to face the center of the simulation area. A feed horn cylinder sits at the focal point. A glow sphere (MeshStandardMaterial with emissive color matching the cell's assigned color) sits at the tower top.

UE Spheres

When setUEPositions() is called with live UE data, each UE is a small sphere (radius ~1.2 units) with a glowing emissive material matching its serving cell's color. A thin cylinder line connects the UE sphere to its serving tower top. UE positions are smoothly interpolated each frame toward their target coordinates using linear interpolation at 8% per frame.

Handover Animation

triggerHandover(fromX, fromZ, toX, toZ) creates two types of animated geometry. First, three concentric expanding rings bloom at the destination cell: RingGeometry objects colored green, gold, and cyan, scaled from near-zero up to a maximum radius and faded out over 1.4 seconds. The rings are offset slightly in Y so they do not Z-fight. Second, a luminous white arc connects source tower to destination tower using QuadraticBezierCurve3 with a peak height of max(55, dist*0.65) units above the midpoint — longer handovers produce taller arcs. The arc uses TubeGeometry wrapped around the curve and fades out over 1.8 seconds.

Cell Load Color Thresholds (loadHex function)

Cell load is mapped to color: load < 0.45 → green (#00ff88), 0.45-0.70 → yellow (#ffcc00), 0.70-0.88 → orange (#ff6600), above 0.88 → red (#ff2244). These same thresholds are used by the loadHex() function in scene.js and the loadColor() function in main.js to color cell card borders and load bars consistently.

8.3 config.js — Static Configuration

ALL_CELLS Array (8 entries)

The ALL_CELLS array defines the 8-cell topology. Index 0 is the LTE macro (id: 'LTE-001', x:0, z:0, initLoad:0.62, color:'#00aaff', type:'lte'). Indices 1-7 are mmWave small cells at various (x, z) coordinates between -92 and +92 units, with initLoads ranging from 0.33 to 0.91 and distinct colors. These coordinates are scene-space coordinates used only for the 3D initial layout — they are overridden by real ns-3 positions once pollBackend() succeeds.

Load Thresholds

LOAD_GREEN=0.45, LOAD_YELLOW=0.70, LOAD_ORANGE=0.88, LOAD_BALANCE_TRIGGER=0.85, LOAD_BALANCE_TARGET=0.60, CRITICAL_LOAD=0.92. These are used by both the 3D scene color logic and the xApp rules display.

xApp Rules

GRU_HANDOVER rule: trigger condition 'cell.load > 0.70', max_concurrent: 2, cooldown_seconds: 30, rssi_threshold_dbm: -100, hysteresis_db: 3, time_to_trigger_ms: 320, inference server at localhost:5000, start_url at localhost:38868, stop_url at localhost:38869. LOAD_BALANCER rule: trigger condition 'cell.load > LOAD_BALANCE_TRIGGER', target condition 'cell.load < LOAD_BALANCE_TARGET', max_ues_to_move: 2.

8.4 ui.js — DOM Components

ui.js provides helper functions that manipulate specific DOM elements. The status panel shows five component status rows (docker, flexric, simulation, pusher, xapp) with colored dots and state text labels. Cell cards are dynamically generated by buildCards() — one card per active cell with a colored load bar, throughput, latency, and UE count. The handover log shows the last 6 handover events with timestamp, UE ID colored in the destination cell's color, and target cell name. The SINR strip shows a canvas sparkline per UE with the most recent 30 SINR samples as a trend line.

8.5 vite.config.js — Proxy Rules and Why They Are Needed

Vite runs on port 3001. The browser's same-origin policy (CORS) blocks a page served from localhost:3001 from making fetch() calls to localhost:8000 or localhost:8001 unless those servers include CORS headers. While the FastAPI backends do include CORSMiddleware, InfluxDB does not. The cleaner solution is to use Vite's built-in proxy: requests to matching paths are transparently forwarded by the Vite dev server itself (server-to-server), bypassing CORS entirely.

Proxy Rule	Forwards To	Path Rewrite	Purpose
/api/*	http://localhost:8000	/api prefix stripped	All 2D backend API calls (/refresh-data, /scenarios, etc.)
/ctrl/*	http://localhost:8001	None (path preserved)	All controller API calls (/ctrl/status, /ctrl/launch-all, etc.)
/xapp-start	http://localhost:38868	Rewritten to /	Direct POST to xApp trigger server start endpoint
/xapp-stop	http://localhost:38869	Rewritten to /	Direct POST to xApp trigger server stop endpoint

CHAPTER 9

Docker Compose — Infrastructure Services

9.1 Overview

The docker-compose.yml at GUI/ defines three services that form the infrastructure layer: influxdb, gui, and grafana. They are started by controller.py's launch-all sequence (only influxdb and gui are started automatically; grafana can be started separately). The Docker Compose file version is 3.8.

9.2 Service: influxdb

- **image: influxdb:1.8-alpine** — Official InfluxDB 1.8 on Alpine Linux. The alpine variant is about 100MB smaller than the Debian-based image. Must be 1.8 not 2.x because the Python influxdb client library uses the v1 HTTP API.
- **env_file: configuration.env** — Injects INFLUXDB_ADMIN_USER=admin and INFLUXDB_ADMIN_PASSWORD=admin and INFLUXDB_DB=influx.
- **ports: 127.0.0.1:8086:8086** — InfluxDB HTTP API bound to localhost only. The data pusher on the host connects to 127.0.0.1:8086 successfully because host networking is used. The '127.0.0.1:' prefix ensures InfluxDB is not exposed to external network interfaces.
- **command** — Custom startup: starts influxd in the background, waits 10 seconds, runs a DELETE to clear stale measurements, then uses 'tail -f /dev/null' to keep the container alive. This initial DELETE clears any leftover data from previous runs stored in the named volume.
- **volumes: influxdb_data:/var/lib/influxdb** — Named volume persists the database directory across container restarts. The volume is also mapped to ./:/imports which allows importing CSV files manually if needed.

9.3 Service: gui (2D Backend)

- **build: . / Dockerfile** — Built from the local Dockerfile in GUI/. This installs the Python requirements (FastAPI, uvicorn, influxdb, paramiko) and copies the application code.
- **ports: 8000:8000** — The 2D backend API is exposed on host port 8000. The Vite proxy routes /api/* to this port.
- **depends_on: influxdb** — Docker Compose starts influxdb before gui. However depends_on only guarantees container start order, not that InfluxDB is ready to accept connections. The 2D backend handles this gracefully by catching InfluxDB connection errors and returning empty data.
- **environment: NS3_HOST=172.17.0.1** — This is the Docker bridge network gateway address — the IP address of the host as seen from inside a Docker container. The 2D backend uses this to reach the host-side controller running on port 8001, and also to discover available scenarios by calling http://172.17.0.1:38866. The value 172.17.0.1 is the default Docker bridge gateway and works on standard Linux Docker installations.

- **environment: INFLUXDB_HOST=influxdb** — Inside the Docker network, services reference each other by service name. The gui container reaches InfluxDB at http://influxdb:8086 (not localhost:8086). This is why the Simulation class reads INFLUXDB_HOST from the environment rather than hardcoding 'localhost'.

9.4 Service: grafana

- **image: grafana/grafana:8.0.2** — Pinned to version 8.0.2 for compatibility with the provisioned dashboard JSON files and the grafana-xyzchart-panel plugin.
- **ports: 3000:3000** — Grafana web UI accessible at http://localhost:3000. Default login: admin/admin.
- **volumes: grafana/provisioning/:/etc/grafana/provisioning/** — Auto-provisions the InfluxDB datasource and dashboard folders at container startup. No manual datasource configuration needed.
- **volumes: grafana/dashboards/:/var/lib/grafana/dashboards/** — Loads the per_Cell_stats.json and per_UE_stats.json dashboard definitions. Grafana reads these from the all.yml provisioning config.

9.5 configuration.env — All Variables Explained

Variable	Value	Used By	Purpose
GF_SECURITY_ADMIN_USER	admin	Grafana	Grafana login username
GF_SECURITY_ADMIN_PASSWORD	admin	Grafana	Grafana login password
GF_INSTALL_PLUGINS	grafana-xyzchart-panel, marcusolsson-csv-datasource	Grafana	Plugins installed at startup for 3D chart and CSV import panels
GF_PANELS_ENABLE_ALPHA	true	Grafana	Enables alpha/experimental panel types
INFLUXDB_DB	influx	InfluxDB	Database created automatically at startup
INFLUXDB_ADMIN_USER	admin	InfluxDB	Admin account username
INFLUXDB_ADMIN_PASSWORD	admin	InfluxDB	Admin account password
INFLUXDB_HOST	influxdb (in compose)	2D backend	Docker service hostname for InfluxDB connection
INFLUXDB_PORT	8086	2D backend	InfluxDB HTTP API port
INFLUXDB_USERNAME	admin	2D backend	InfluxDB auth username
INFLUXDB_PASSWORD	admin	2D backend	InfluxDB auth password
INFLUXDB_DATABASE	influx	2D backend	Database name to query

Variable	Value	Used By	Purpose
NS3_HOST	172.17.0.1	2D backend	Docker bridge gateway IP to reach the host

9.6 Manual Docker Commands

Task	Command
Start all services	cd GUI/ && docker compose up -d
Start only InfluxDB and 2D backend	docker compose up -d influxdb gui
Stop and remove containers	docker compose down
Stop and delete volumes (full reset)	docker compose down --volumes
View logs for 2D backend	docker compose logs -f gui
View InfluxDB logs	docker compose logs -f influxdb
Shell into InfluxDB container	docker exec -it $(docker ps -q -f name=influxdb) influx
Restart just the 2D backend	docker compose restart gui
Rebuild 2D backend image	docker compose build gui && docker compose up -d gui

Important: docker compose down --volumes permanently deletes all InfluxDB historical data stored in the influxdb_data volume. This does NOT affect sim_decisions.db (SQLite) or the 3D_GUI_Sim_Results/ directory — those live on the host filesystem and are not inside any Docker volume.

CHAPTER 10

gru.sh and kill_sim.sh — Launch and Kill Scripts

10.1 Overview and Design Philosophy

The launch and kill scripts exist because the O-RAN simulation platform involves eight separate processes that must be started and stopped in a specific order with specific timing delays between them. Doing this manually from eight terminal windows is error-prone and tedious. gru.sh encodes the correct launch sequence into a single repeatable command. kill_sim.sh encodes the correct shutdown sequence.

Both scripts are designed to be idempotent: running gru.sh when some components are already running will not cause errors — the cleanup step at the beginning kills everything before starting fresh. Running kill_sim.sh when nothing is running is also safe — the pkill and fuser commands simply report 'no such process' and continue.

The scripts must be kept in sync with controller.py. Any time you change a process name, a port number, or a file path in controller.py, the corresponding change must be reflected in both gru.sh and kill_sim.sh. This is noted in the project memory file as a required maintenance rule: 'After ANY fix, always update MANUAL_COMMANDS.txt and gru.sh if affected.'

10.2 gru.sh — Arguments and Defaults

gru.sh accepts three optional positional arguments that control the simulation parameters. All have sensible defaults so the script can be called with no arguments for a standard 60-second test run.

Argument	Position	Default	NS-3 Flag	Description
simTime	1	60	--simTime	Duration of the ns-3 simulation in seconds. Longer runs give more handover data but take proportionally more time.
nUEs	2	20	--N_Ues	Number of User Equipment instances. More UEs means more handovers and more KPM measurement points, but also more InfluxDB writes.
nCells	3	7	--N_MmWaveEnbNodes	Number of mmWave small cell gNBs in addition to the single LTE macro cell (which is always present). Value 7 gives the standard 8-cell topology. Value 1 gives 2 cells total (minimum for handover testing).

```
bash gru.sh # defaults: 60s, 20 UEs, 7 mmWave cells
bash gru.sh 60 20 7 # same as defaults, explicit
bash gru.sh 120 30 8 # 120s sim, 30 UEs, 8 cells
bash gru.sh 30 10 3 # quick 30s test with 10 UEs, 3 small cells
```

1. **Cleanup stale processes.** The script starts by killing every process that might be left from a previous run: nearRT-RIC, ns3.42 binaries, sim_data_pusher.py, gru_xapp.py, and any xapp_rc_handover_ctrl binary. It also kills any process listening on ports 5000, 3001, and 8001 using fuser -k. This step is not optional — skipping it and starting FlexRIC while a previous FlexRIC instance is still bound to port 36421 will cause the new instance to fail silently.
2. **Start Docker services.** Runs 'docker compose up -d influxdb gui' from the GUI/ directory. This starts InfluxDB and the 2D backend in daemon mode. Both must be up before ns-3 can push data and before the 3D frontend can display anything meaningful.
3. **Wait 3 seconds.** A brief sleep to allow Docker containers to initialise their network interfaces and bind ports. InfluxDB takes 2-3 seconds before its HTTP API becomes responsive.
4. **Start FlexRIC nearRT-RIC controller.** Spawns the nearRT-RIC binary in the background with its configuration file. FlexRIC must start before ns-3 because ns-3 initiates the E2 SCTP connection to FlexRIC — if FlexRIC is not listening on port 36421 when ns-3 starts, the connection will be refused and the xApp will never receive KPM reports.
5. **Wait 15 seconds, polling /ctrl/status.** The script polls http://localhost:8001/ctrl/status every 3 seconds for up to 15 seconds waiting for FlexRIC to show as running. This is more reliable than a fixed sleep because on a slow machine FlexRIC may take longer than 3 seconds to start.
6. **Start the 3D frontend Vite dev server.** Runs 'npm run dev' from the 5g-gui-v2/ directory in the background. This is the UI server at http://localhost:3001. Starting it here means by the time the simulation is running the UI is already available.
7. **Wait 4 seconds.** Allows Vite to finish its startup compilation and begin serving requests before the controller is launched.
8. **POST /ctrl/launch-all.** Sends the launch-all HTTP request to the controller on port 8001. The controller then handles all remaining steps autonomously: starting the GRU inference service, ns-3, the data pusher, and the xApp in sequence with proper E2 connection waiting. The gru.sh script returns control to the terminal after this POST — it does not wait for the simulation to finish.
9. **Controller orchestrates everything.** From this point forward the controller's _launch_all_task coroutine is running in the background. It starts GRU service, waits for ns-3 to connect all gNBs to FlexRIC, starts the data pusher, starts the xApp, waits for simulation completion, and calls save_sim_results() automatically.
10. **Auto-save at end.** When ns-3 exits naturally (simTime reached), the controller calls save_sim_results() which archives handover.csv, generates the four plots, writes decision_log.csv and decision_summary.json, and inserts all decisions into the SQLite database.

Key Point: ORDER MATTERS: Docker must be running before the data pusher starts (pusher writes to InfluxDB). FlexRIC must be running before ns-3 starts (ns-3 initiates E2 connections). The xApp must start AFTER ns-3 has connected all gNBs (otherwise the xApp will send RC commands with no E2 sessions to deliver them to).

10.2 kill_sim.sh — 5-Step Shutdown Sequence

kill_sim.sh is the complementary script that shuts down everything cleanly. It is designed to work whether the simulation finished naturally or is still running. After killing processes, it automatically generates the

post-simulation analysis plots.

1. **Kill simulation processes.** Sends SIGKILL (-9) to all ns-3 binaries (pgrep -f gru_scenario, pgrep -f ns3.42), the data pusher, the GRU inference service, and the xApp binary. SIGKILL is used instead of SIGTERM because ns-3 sometimes ignores SIGTERM when deep in its simulation event loop.
2. **Kill controller, Vite frontend, and free ports 3001 and 8001.** Kills the controller.py process (which frees port 8001) and the Vite dev server (which frees port 3001). Uses 'fuser -k 3001/tcp' and 'fuser -k 8001/tcp' as belt-and-suspenders to ensure those ports are definitely free for the next launch.
3. **Stop Docker services.** Runs 'docker compose down' in the GUI/ directory. This stops and removes the influxdb, gui, and grafana containers. The InfluxDB data volume is preserved (not deleted) because 'down' without '--volumes' does not touch named volumes.
4. **Free ports 8000, 8086, 3000, and 5000.** Runs fuser -k for each of these ports. Port 8000 is the 2D backend, 8086 is InfluxDB, 3000 is Grafana, 5000 is the GRU Flask inference service. Even after docker compose down the ports may still appear occupied by lingering network namespaces — fuser -k clears them immediately.
5. **Auto-generate missing plots.** Calls 'python3 generate_plots.py' in auto-detect mode. The script scans 3D_GUI_Sim_Results/ for the most recently modified sim directory and generates any missing plots and analysis files. This step is idempotent — if plots already exist they are regenerated (overwritten), not duplicated.

Important: kill_sim.sh does NOT call save_sim_results() — it only calls generate_plots.py. If the controller did not save results before being killed, handover.csv will NOT be archived to a sim directory. In that case, generate_plots.py will process whichever sim directory was most recently modified, which may not be the current run. For a guaranteed clean save, always let the simulation finish naturally via gru.sh.

10.3 Why Order Matters — Dependency Chain

The startup order is a hard constraint imposed by the protocol architecture of the system. Each component communicates with others via network sockets, and sockets fail immediately if the remote endpoint is not listening. The table below documents every dependency and the failure mode if it is violated.

Component	Depends On	Failure If Started Too Early
Docker (InfluxDB)	Nothing	No failure — first to start
FlexRIC nearRT-RIC	Nothing (but before ns-3)	Port 36421 not open when ns-3 tries to connect → E2 connection refused → no KPM reports
GRU Flask service	Nothing (but before xApp)	xApp sends HTTP to localhost:5000 and gets connection refused → inference fails → no handovers
ns-3 simulation	FlexRIC running + Docker (InfluxDB)	E2 handshake fails → KPM data never reaches InfluxDB → GUI shows empty
sim_data_pusher.py	InfluxDB reachable + ns-3 writing CSVs	Writes to non-existent DB → client.write_points raises InfluxDBClientError

Component	Depends On	Failure If Started Too Early
xApp (xapp_handover_gru)	ns-3 connected to FlexRIC (all gNBs)	RC commands sent with no active E2 sessions → silently dropped → no handovers

10.4 Timing Budget for a Standard Run

The following timeline shows the wall-clock time for each phase of a standard 60-second simulation run when using gru.sh with default parameters. Times are approximate and depend on CPU speed and Docker pull cache status.

Wall-Clock Time	Phase	What Is Happening
0:00 – 0:05	Cleanup	pkill commands, fuser port kills, log truncation
0:05 – 0:15	Docker start	docker compose up -d influxdb gui, waiting for port 8086
0:15 – 0:30	FlexRIC start	nearRT-RIC spawned, waiting for port 36421 to open
0:30 – 0:34	Vite start	npm run dev, waiting for Vite to compile assets
0:34 – 0:38	Controller launch	POST /ctrl/launch-all, controller starts GRU service
0:38 – 0:55	NS-3 E2 connect	ns-3 starts, 7 gNBs each complete E2 Setup with FlexRIC
0:55 – 1:00	Pusher + xApp	sim_data_pusher and xapp_handover_gru spawned
1:00 – 2:00	Simulation running	60 seconds of ns-3 sim time; GUI live; xApp making handover decisions
2:00 – 2:10	Save results	save_sim_results(): copy files, detect PP, write SQLite, generate plots
2:10+	Complete	All processes idle; results in 3D_GUI_Sim_Results/

10.5 Environment Variables Used by gru.sh

gru.sh reads several hardcoded paths. If you move the project directory or change the ns-3 build directory structure, these paths must be updated in the script.

Variable / Path	Default Value	Purpose
GUI_DIR	yousef_fathy/ns-O-RAN-flexric/mmwave-LENA-oran/GUI	Directory containing docker-compose.yml
NS3_DIR	yousef_fathy/ns-O-RAN-flexric/mmwave-LENA-oran/ns3-mmwave-oran	NS-3 build root, where ./ns3 is located
FLEXRIC_BIN	build/examples/ric/nearRT-RIC	Relative path to FlexRIC binary from NS3_DIR
CONTROLLER_PORT	8001	Port the FastAPI controller listens on

Variable / Path	Default Value	Purpose
GRU_PORT	5000	Port the GRU Flask inference service listens on
VITE_PORT	3001	Port Vite dev server listens on

10.6 Common gru.sh Failure Modes and How to Diagnose

The following are the most common reasons gru.sh fails mid-launch:

- **Docker fails to start (step 2):** Usually caused by Docker daemon not running ('Cannot connect to the Docker daemon'). Fix: 'sudo systemctl start docker'. Or caused by configuration.env missing — fix by checking the GUI/ directory.
- **FlexRIC never reaches LISTEN on 36421 (step 5):** FlexRIC crashed at startup due to a missing flexric.conf file or a shared library not found. Check /tmp/flexric.log for 'error' or 'No such file' messages.
- **launch-all POST fails (step 8):** The controller is not running. Run 'python3 controller.py &' manually from the 5g-gui-v2/ directory, then retry.
- **Vite fails to start (step 6):** node_modules is missing. Run 'npm install' from the 5g-gui-v2/ directory first.

Important: If gru.sh is interrupted (Ctrl+C) after step 8, the controller's launch-all background task is still running. The simulation will continue and auto-save. To stop it cleanly, run kill_sim.sh or POST to /ctrl/stop-all.

CHAPTER 11

Complete Data Flow — Every Step

This chapter traces the complete lifecycle of a single data point — from the moment ns-3 produces a KPM measurement through every component until it appears on the 3D GUI screen. It also traces the parallel xApp decision path. Understanding this flow is essential for debugging 'why isn't the GUI updating?' questions.

Phase 1 — NS-3 Generates Data

Every 0.05 simulation seconds (the indicationPeriodicity parameter), the ns-3 gru_scenario C++ code writes a batch of KPM rows to its output CSV files. The files written are: ue_position.txt (UE coordinates and serving cell), du-cell-N.txt for each mmWave gNB (PRB usage, active UEs, latency), cu-cp-cell-N.txt (SINR per UE per cell), and cu-up-cell-N.txt (throughput, PDCP volume). These files accumulate rows continuously for the entire simulation duration.

When the xApp triggers a handover (via an RC control message), ns-3 executes the handover at the physical layer and appends one row to handover.csv with the format: time_sec, ue_id, from_cell, to_cell, event, executed_ok. The executed_ok field is set to 1 if the handover succeeded, 0 if ns-3 rejected it (e.g. if the UE was already being handed over).

> ***Note:*** *handover.csv is reset to an empty header before each launch-all run. Without this reset, rows from a previous run would contaminate the ping-pong detection for the new run. The reset happens in step 3 of the launch-all task (the GRU service start step).*

Phase 2 — sim_data_pusher Reads and Pushes

sim_data_pusher.py runs a 3-second loop. On each iteration it calls read_and_clear_csv() for every discovered CSV file. read_and_clear_csv() reads all current rows, immediately rewrites the file with only the header row (clearing it), and returns the rows it read. This prevents the pusher from re-processing old rows on the next cycle — each row is pushed exactly once.

The pusher then builds InfluxDB line protocol points from the rows and calls client.write_points() in a single batch. Each point has a measurement name (e.g. 'ue_5_l3 serving sinr'), a timestamp (current wall-clock time), and a single field named 'value'. InfluxDB stores these as a time series. After the write, the pusher sleeps 3 seconds and repeats.

Phase 3 — 2D GUI Backend Queries InfluxDB

Every time the Vite frontend polls /api/refresh-data (every 1.5 seconds from the browser), the 2D backend's SimulationManager.refresh_simulation() method creates a new Simulation object. The Simulation.__init__ method immediately calls get_simulation_data() which issues a SELECT * FROM {measurement} LIMIT 1 query for every known UE field and every known cell field.

The results are assembled into a list of Ue dataclass instances and Cell dataclass instances. The data_controller then serialises these to JSON and returns the response: {ues: [...], cells: [...], max_x_max_y: [x, y], simulation_status: ..., energy: ...}. This JSON is the only source of live data for the 3D frontend.

Phase 4 — 3D Frontend Processes the Response

main.js receives the /refresh-data JSON in its pollBackend() function. It extracts max_x and max_y from max_x_max_y[0] to build the ns3ToScene() coordinate scaling function: scene_x = (ns3_x / max_x) * SCENE_SCALE. This maps the ns-3 meter coordinates to Three.js scene units.

For each cell in the response, the load value (dlPrbUsage_percentage / 100) is updated in NET.cells, and the cell card DOM element is updated with the new PRB percentage, active UE count, throughput, and latency values. The cell card border color is updated by loadColor() based on the four threshold values.

For each UE in the response, the serving cell ID is compared against prevServingCells[ueIndex]. If the cell ID has changed and both old and new values are non-zero, a handover event is triggered: triggerHandover() creates the expanding ring + Bezier arc animation in the 3D scene, flashUELabel() brightens the UE sphere, and logHandover() appends an entry to the on-screen handover log panel. prevServingCells[ueIndex] is updated to the new cell ID.

The UE sphere positions are updated via setUEPositions() in scene.js. Each UE sphere's target position is set to ns3ToScene(ue.x_position, ue.y_position). The render loop interpolates each sphere toward its target at 8% per frame, creating a smooth motion effect rather than a jarring teleport.

Phase 5 — xApp Decisions (Parallel Path)

The xApp data flow runs completely in parallel with the GUI data flow. It does not go through InfluxDB or the 2D backend — it operates entirely in the FlexRIC E2 interface layer.

- **KPM reports:** Every 0.05 simulation seconds, ns-3 sends E2 KPM Indication messages over the SCTP connection to FlexRIC on port 36421. Each message contains the current SINR values for each UE served by that gNB.
- **xApp C code receives KPM:** The xapp_handover_gru binary (registered with FlexRIC as a KPM subscriber) receives each Indication via FlexRIC's RAN function callback. It extracts the UE SINR values and neighbor cell SINRs, and assembles a feature vector.
- **GRU inference service:** The xApp sends an HTTP POST to http://localhost:5000/predict with the feature vector as JSON. The GRU Flask service passes the vector through the trained GRU model and returns the predicted best target cell index.
- **RC control message:** If the predicted target cell differs from the UE's current serving cell and the SINR difference exceeds hoSinrDifference (3 dB default), the xApp constructs an E2 RC Control message and sends it to FlexRIC. FlexRIC forwards the RC message to the target gNB's E2 agent in ns-3.
- **NS-3 executes handover:** The E2 agent in ns-3 receives the RC message and calls the handover function at the MAC layer. NS-3 performs the full RRC handover procedure and writes a row to handover.csv with executed_ok=1.

Phase 6 — Post-Simulation Auto-Save

When ns-3 exits (simTime reached), the launch-all task's polling loop (step 7: 'wait for simulation to finish') detects that _alive('simulation') has returned False. It immediately calls save_sim_results(tag=scenario). save_sim_results() copies all output files to a new numbered sim directory, applies the ping-pong detection algorithm, writes the decision log to SQLite, and generates the four matplotlib plots. The result dict is stored in

_last_result and exposed by GET /ctrl/last-result for the frontend to display.

> **Key Point:** The entire flow from ns-3 CSV write to 3D GUI update takes at most 3 + 1.5 = 4.5 seconds: up to 3 seconds for the pusher's poll cycle to pick up new rows, plus up to 1.5 seconds for the frontend to poll /refresh-data. In practice the average delay is ~2.25 seconds.

11.7 Data Flow Timing Diagram

The following table shows the timing of each component's activity during a single 3-second pusher cycle, starting from when ns-3 writes a CSV row.

Time Offset	Component	Action
T + 0.00s	NS-3 C++	Writes new KPM rows to du-cell-N.txt and ue_position.txt at indicationPeriodicity boundary
T + 0.00s	NS-3 C++	If xApp sent RC command: appends row to handover.csv with executed_ok=1
T + 0.00 to 3.0s	sim_data_pusher	Sleeping (3-second poll cycle) — rows accumulate in CSV files
T + 3.0s	sim_data_pusher	Wakes up, calls read_and_clear_csv() for all discovered files
T + 3.01s	sim_data_pusher	Calls client.write_points() with batch of all new rows
T + 3.01s	InfluxDB	Receives write batch, indexes measurements by timestamp
T + 3.0 to 4.5s	2D backend	Next /refresh-data poll fires (1.5s interval from last poll)
T + 4.5s	2D backend	Issues SELECT * FROM {measurement} LIMIT 1 for all fields
T + 4.5s	InfluxDB	Returns most-recent point for each queried measurement
T + 4.51s	2D backend	Assembles Ue and Cell dataclasses, returns JSON response
T + 4.51s	3D frontend (main.js)	Receives JSON, runs handover detection, updates NET state
T + 4.52s	3D frontend (scene.js)	setUEPositions() called; UE sphere targets updated
T + 4.52s	3D frontend (ui.js)	Cell card DOM elements updated with new load/SINR values
T + 4.52s	3D frontend (renderer)	Next animation frame interpolates UE spheres toward targets

11.8 Data Flow Comparison — GUI Path vs xApp Path

It is important to understand that the GUI data path and the xApp decision path are completely independent pipelines. The GUI never influences handover decisions. The xApp never reads from InfluxDB. They share only the ns-3 simulation as a common source of truth.

Dimension	GUI Data Path	xApp Decision Path
Protocol	CSV file → Python → HTTP → JSON → JavaScript	SCTP E2 → C binary → HTTP → Python → SCTP E2 → C++

Dimension	GUI Data Path	xApp Decision Path
Latency	3 – 4.5 seconds end-to-end	< 100 milliseconds end-to-end
Purpose	Human situational awareness and visualisation	Automated handover decision execution
Data format	InfluxQL time-series points, JSON REST	ASN.1 E2AP SCTP messages, HTTP JSON
Failure impact	GUI goes blank; simulation continues unaffected	No handovers fire; simulation continues with degraded performance
Key files	sim_data_pusher.py, GUI/main.py, src/main.js	xapp_handover_gru.c, gru_xapp.py, controller.py launch-all
Databases used	InfluxDB (write + read), SQLite (write only)	None — pure in-memory processing

11.9 Frequently Asked Questions

Q: Why does the GUI sometimes show UEs at position (0, 0)?

A: The ue_position.txt file has not been pushed to InfluxDB yet — either the pusher just started and has not completed its first cycle, or ns-3 has not written the file yet. Wait 3-5 seconds after the simulation starts for the first position data to appear.

Q: Why do cell loads stay at 0% even when the simulation is running?

A: The du-cell-N.txt files are being read but the 'dlprbusage' field is zero. This can happen if the ns-3 build was compiled without the du-cell reporting hook, or if the cell has no active UEs (all 20 UEs happen to be served by other cells at that moment).

Q: Why does the handover log in the GUI sometimes show handovers that are not in handover.csv?

A: The GUI's handover detection in main.js is based on comparing consecutive /refresh-data responses. If a UE's serving cell changes and then changes back within a single 1.5-second poll interval (two rapid handovers), both events appear in the GUI log. However, handover.csv may show both, neither, or one depending on ns-3 execution timing relative to the CSV write cycle.

Q: Can I run the GUI without the simulation running?

A: Yes. If no simulation is running, InfluxDB will return empty results for all queries. The 2D backend will return an empty ues list and cells at 0% load. The 3D scene will show towers but no UE spheres. The system control panel will show all components as OFFLINE.

CHAPTER 12

API Quick Reference — All Routes

This chapter lists every HTTP route exposed by the system in a single reference table. The controller runs on port 8001 (host, not Docker). The 2D backend runs on port 8000 (inside Docker, proxied through Vite at /api/*). All controller routes can be called directly with curl or from the 3D frontend via the Vite /ctrl/* proxy.

12.1 Controller Routes (port 8001, prefix /ctrl)

Method	Path	Request Body	Response	Purpose
GET	/ctrl/status	—	{docker, flexric, simulation, pusher, xapp: bool}	Component health check; used by status panel
GET	/ctrl/scenarios	—	["gru_scenario", ...]	List .cc scenario files in scratch/
GET	/ctrl/logs/{component}	?lines=40	{lines: [str, ...]}	Tail last N lines of component log file
GET	/ctrl/decisions	?sim=sim005&limit;=500	{decisions: [{uuid, sim, time_sec, ue_id, from_cell, to_cell, is_correct, saved_at}]}	Query SQLite decisions table
GET	/ctrl/last-result	—	{folder, sim, pp_count, pp_rate_pct, accuracy_pct} or {status: 'no results yet'}	Last save_sim_results() output
POST	/ctrl/docker/start	—	{status, output}	docker compose up -d influxdb gui
POST	/ctrl/docker/stop	—	{status}	docker compose down
POST	/ctrl/flexric/start	—	{status: 'started'|'already_running'}	Spawn nearRT-RIC binary

Method	Path	Request Body	Response	Purpose
POST	/ctrl/flexric/stop	—	{status: 'killed'}	Kill nearRT-RIC + pkill
POST	/ctrl/simulation/start	{scenario, n_ues, n_mmwave, sim_time, e2_term_ip}	{status: 'started'|'already_running'}	Spawn ns-3 simulation with params
POST	/ctrl/simulation/stop	—	{status: 'killed'}	Kill ns-3 process group + pkill -9
POST	/ctrl/pusher/start	—	{status: 'started'}	Spawn sim_data_pusher.py
POST	/ctrl/pusher/stop	—	{status: 'killed'}	Kill pusher
POST	/ctrl/xapp/start	—	{status: 'started'|'error'}	Start xApp via trigger server
POST	/ctrl/xapp/stop	—	{status: 'stopped'}	Stop xApp via trigger server + pkill
POST	/ctrl/launch-all	{scenario, n_ues, n_mmwave, sim_time}	{status: 'launching'}	Start full 10-step async background launch
POST	/ctrl/stop-all	—	{status: 'stopped'}	Emergency kill all processes
GET	/ctrl/save-results	?tag=gru_scenario	{folder, sim, pp_count, accuracy_pct, ...}	Manually trigger save_sim_results()

12.2 2D Backend Routes (port 8000, proxied via /api)

Method	Path	Request Body	Response	Purpose
GET	/refresh-data	—	{ues: [...], cells: [...], max_x_max_y: [x,y], simulation_status, energy}	Main polling endpoint — all live KPM data

Method	Path	Request Body	Response	Purpose
GET	/scenarios	—	[{name, description}]	Available simulation scenarios list
GET	/health	—	{status: 'ok'}	Docker health-check endpoint
POST	/start	{scenario_name}	{status}	Start simulation via 2D backend (legacy path)
POST	/stop	—	{status}	Stop simulation via 2D backend (legacy path)

12.3 GRU Inference Service Routes (port 5000)

Method	Path	Request Body	Response	Purpose
GET	/health	—	{status: 'ok', model: 'gru'}	Check if GRU Flask service is running
POST	/predict	{features: [float, ...]}	{target_cell: int, confidence: float}	Run GRU inference; return predicted best cell

12.4 Quick curl Examples

```
# Check all component status
curl http://localhost:8001/ctrl/status

# Launch simulation (60s, 20 UEs, 7 cells, gru_scenario)
curl -X POST http://localhost:8001/ctrl/launch-all \
-H "Content-Type: application/json" \
-d '{"scenario":"gru_scenario","n_ues":20,"n_mmwave":7,"sim_time":60}'

# Get live UE and cell data
curl http://localhost:8000/refresh-data | python3 -m json.tool | head -60
```

```
# Get last 20 lines of FlexRIC log
curl 'http://localhost:8001/ctrl/logs/flexric?lines=20'

# Get decisions for sim007
curl 'http://localhost:8001/ctrl/decisions?sim=sim007'
```

12.5 Request and Response Details for Key Endpoints

POST /ctrl/launch-all — Full Request and Response Example

Request body (all fields optional with defaults as shown):

```
POST http://localhost:8001/ctrl/launch-all Content-Type: application/json {
"scenario": "gru_scenario", "n_ues": 20, "n_mmwave": 7, "sim_time": 60 }
```

Immediate response (task starts in background):

```
HTTP 200 OK { "status": "launching" }
```

The actual simulation start is asynchronous. Poll GET /ctrl/status to track component state changes. Poll GET /ctrl/last-result after the simulation ends to retrieve accuracy and ping-pong statistics.

GET /api/refresh-data — Full Response Example

This is the most data-rich endpoint. A typical condensed response:

```
HTTP 200 OK { "ues": [ { "ue_id": 1, "x_position": 342.5, "y_position": 1203.8,
"MMWave_Cell": 3, "L3servingSINR_dB": 18.4 }, ... (20 total for default run) ],
"cells": [ { "cell_id": 3, "dlPrbUsage_percentage": 74.2, "MeanActiveUEsDownlink":
5.0, "x_position": 800.0, "y_position": 600.0, "average_latency_ms": 1.8 }, ... (8
total for default 7-mmWave run) ], "max_x_max_y": [3000.0, 3000.0],
"simulation_status": "running", "energy": { "total_joules": 0.0, "per_cell": {} } }
```

GET /ctrl/status — Component Status Details

How each boolean field is computed internally:

```
{ "docker": true, // docker compose ps --filter status=running "flexric": true, //
nearRT-RIC in _procs and poll() is None "simulation": true, // pgrep -f
'ns3.42-gru_scenario' returns match "pusher": true, // sim_data_pusher.py in _procs
and alive "xapp": true // xapp_handover_gru found by ps aux }
```

12.6 Error Response Formats

The controller uses consistent error response formats across all endpoints.

- **Component already running:** Returns HTTP 200 with {'status': 'already_running'} rather than HTTP 409. This allows scripts to call start endpoints idempotently without checking status first.
- **Process not found:** Returns {'status': 'not_found'} if a kill endpoint is called for a component that was never started or was already killed.
- **Internal error:** FastAPI returns HTTP 500 with a detail field if an unhandled exception occurs. The full Python traceback appears in the component log file. Check /tmp/farouk_*.log for details.
- **Log file missing:** GET /ctrl/logs/{component} returns {'lines': []} rather than HTTP 404 if the log file does not exist (component never started).

12.7 Authentication and Security Notes

The current system has no authentication on any endpoint. This is intentional for a development/research platform running on localhost. All services bind to 127.0.0.1 or 0.0.0.0 with the assumption that external access is blocked by the host firewall. InfluxDB explicitly binds to 127.0.0.1:8086 in the docker-compose.yml to prevent external access from other machines. If deploying on a machine with public network interfaces, add firewall rules to block ports 8000, 8001, 8086, 3001, and 3000 from external access, or add API key authentication to the FastAPI applications.

> ***Important: The controller.py CORSMiddleware uses allow_origins=['*'] which permits any website to make requests to the controller from a user's browser. This is acceptable on localhost but must be restricted to specific origins before any public or shared-machine deployment.***

CHAPTER 13

Developer Guide — How to Add New Features

This chapter provides step-by-step guides for the four most common types of extensions developers make to this system. Each guide is independent and self-contained.

13.1 Add a New API Route to the Controller

Example: add GET /ctrl/sim-count that returns the number of completed simulation directories in 3D_GUI_Sim_Results/.

1. **Define the handler function** in controller.py. Place it near other GET handlers. Use FastAPI's async def if the handler does I/O.

```
@app.get('/ctrl/sim-count') async def get_sim_count(): import pathlib count = len([d for d in pathlib.Path(RESULTS_DIR).iterdir() if d.is_dir() and d.name[:3] == 'sim']) return {'count': count}
```

2. **Restart the controller** for the route to be registered. Kill the current controller process and start a fresh one, or use the kill_sim.sh + gru.sh cycle.
3. **Update vite.config.js** if the route needs to be accessible from the browser frontend. Routes under /ctrl/* are already proxied. If you add a route under a new prefix (e.g. /metrics/*), add a new proxy entry in the server.proxy section of vite.config.js.
4. **Test with curl** before adding frontend code.

```
curl http://localhost:8001/ctrl/sim-count
```

5. **Call from JavaScript** with fetch('/ctrl/sim-count').

13.2 Add a New Metric to 3D Cell Cards

Example: add a 'buffer_occupancy' field that shows the DU buffer fullness percentage on each cell card.

1. **Add the InfluxDB query** in GUI/src/simulation_objects/simulation.py. In get_simulation_data(), find the loop that builds Cell objects. Add a call to get_last_value_from_measurement(f'du-cell-{cell_id}_buffer_occupancy') and store the result.
2. **Add the field to the Cell dataclass** in cell.py.

```
@dataclass class Cell: # ... existing fields ... buffer_occupancy: float = 0.0
```

3. **Include it in the /refresh-data response.** Since Cell is a dataclass, it auto-serialises to dict — no changes to data_controller.py needed as long as the field is on the dataclass.
4. **Add the DOM element** to the cell card HTML template in 5g-gui-v2/src/ui.js inside the buildCards() function. Add a span with an id like 'cell-buffer-{id}' inside the card HTML string.
5. **Update the card** in the updateCards() function in ui.js. Find the cell in the cells array by cell_id and set the innerHTML of the span.

```
document.getElementById(`cell-buffer-${cell.cell_id}`).textContent = (cell.buffer_occupancy ?? 0).toFixed(1) + '%';
```

13.3 Add a New Plot

Example: add a 'ho_cell_matrix.png' heatmap showing how many handovers occurred between each pair of cells.

1. **Add the plot function** in generate_plots.py. The function receives the decisions list (each decision is a dict with from_cell, to_cell, time_sec, ue_id, is_correct).

```
def plot_cell_matrix(decisions, plots_dir): import numpy as np cells =
sorted({d['from_cell'] for d in decisions} | {d['to_cell'] for d in decisions}) idx =
{c: i for i, c in enumerate(cells)} mat = np.zeros((len(cells), len(cells))) for d in
decisions: mat[idx[d['from_cell']]][idx[d['to_cell']]] += 1 fig, ax =
plt.subplots(figsize=(8, 7)) im = ax.imshow(mat, cmap='Blues')
ax.set_xticks(range(len(cells))); ax.set_xticklabels(cells)
ax.set_yticks(range(len(cells))); ax.set_yticklabels(cells) ax.set_xlabel('To
Cell'); ax.set_ylabel('From Cell') ax.set_title('Handover Cell Transition Matrix')
plt.colorbar(im, ax=ax) plt.tight_layout() plt.savefig(os.path.join(plots_dir,
'ho_cell_matrix.png'), dpi=150) plt.close()
```

2. **Call the function** inside _generate_plots() (in controller.py) or inside the main block of generate_plots.py, right after the existing plot calls. Pass the decisions list and the plots_dir path.
3. **Test it** by running generate_plots.py on an existing sim directory.

```
python3 /home/omar_farouk/open-ran-clean/5g-gui-v2/generate_plots.py \ /home/omar_fa
rouk/open-ran-clean/3D_GUI_Sim_Results/sim010_20260505_210259_gru_scenario
```

13.4 Add a New Simulation Scenario

Example: add a mobility-heavy scenario called 'highway_scenario' with faster UE movement.

1. **Create the C++ scenario file** in the ns-3 scratch directory at yousef_fathy/ns-O-RAN-flexric/mmwave-LENA-oran/ns3-mmwave-oran/scratch/highway_scenario.cc. This file can be copied from gru_scenario.cc as a starting point. Modify the UE mobility model parameters (e.g. increase RandomWaypointMobilityModel speed) and the E2 function IDs if needed.
2. **The scenario automatically appears** in GET /ctrl/scenarios because that endpoint scans scratch/ for .cc files. No controller changes needed. It will also appear in the 3D GUI's scenario dropdown on the next page load.
3. **To launch with the new scenario** from gru.sh, pass the scenario name as the first argument (if you modify gru.sh) or use the GUI dropdown. The launch-all endpoint accepts the scenario name in the request body and passes it to ./ns3 run scratch/{scenario}.cc.

Important: New scenarios must implement the same E2 KPM and RC function IDs (KPM_E2functionID=2, RC_E2functionID=3) as gru_scenario.cc, or the xApp will not be able to subscribe to KPM reports or send RC control messages. If you change the function IDs, update them in both the .cc file and the xApp source.

13.5 Modify the Ping-Pong Detection Window

Example: change the ping-pong detection window from 5 seconds to 10 seconds to be more lenient about what counts as a harmful oscillation.

1. **Locate the constant** in generate_plots.py. Search for '5.0' — there is a single comparison 'float(rows[i]["time_sec"]) - float(rows[i-1]["time_sec"]) <= 5.0'. Change 5.0 to 10.0.

2. **Update the same value** in controller.py's _calc_pingpong() and _build_decision_log() functions. Both functions have the same 5-second window check. Changing one without the other will cause generate_plots.py and the controller's live save_sim_results() to disagree on ping-pong counts.
3. **Regenerate plots** for existing runs if you want them to use the new window. Run generate_plots.py on each sim directory. The SQLite database will NOT be updated automatically — you must delete and re-save each sim's data if historical accuracy needs to be consistent.

13.6 Add a Health-Check Webhook

Example: send a notification to a Slack webhook URL when the simulation finishes and the accuracy is below 80%.

1. **Add a Slack webhook URL** as a constant at the top of controller.py:

```
SLACK_WEBHOOK = os.getenv('SLACK_WEBHOOK', '')
```

2. **Add the notification call** at the end of save_sim_results(), after the result dict is populated:

```
if SLACK_WEBHOOK and result.get('accuracy_pct', 100) < 80: import requests as _req
_req.post(SLACK_WEBHOOK, json={ 'text': f'Sim {result["sim"]} finished. ' f'Accuracy:
{result["accuracy_pct"]:.1f}% (below 80% threshold)' }, timeout=5)
```

3. **Set the environment variable** before starting the controller:

```
export SLACK_WEBHOOK=https://hooks.slack.com/services/T.../B.../...
python3 controller.py &
```

4. **Test with a dummy run** or by calling /ctrl/save-results on an existing sim directory that has low accuracy.

13.7 Common Code Patterns Used Throughout the Codebase

These patterns appear repeatedly and are worth understanding before extending any module:

subprocess.Popen with process group (controller.py)

```
proc = subprocess.Popen( cmd, cwd=NS3_DIR, stdout=log_fh, stderr=log_fh,
preexec_fn=os.setsid # create new process group ) # Kill the entire group:
os.killpg(os.getpgid(proc.pid), signal.SIGTERM)
```

FastAPI BackgroundTask for long operations (controller.py)

```
@app.post('/ctrl/launch-all') async def launch_all(params: SimParams, bg:
BackgroundTasks): bg.add_task(_launch_all_task, params) return {'status':
'launching'}
```

InfluxDB LIMIT 1 query pattern (simulation.py)

```
def get_last_value_from_measurement(self, measurement: str): result =
self.client.query( f'SELECT * FROM "{measurement}" LIMIT 1' ) points =
list(result.get_points()) if points: return points[0].get('value', 0.0) return 0.0
```

Dataclass serialisation to JSON (data_controller.py)

```
# Dataclasses auto-convert to dict via asdict(): from dataclasses import asdict
return JSONResponse({'ues': [asdict(u) for u in ues], 'cells': [asdict(c) for c in
cells]})
```

CHAPTER 14

Troubleshooting — 10 Common Issues

Each issue below is presented with: the symptom you observe, the diagnosis command to confirm the root cause, and the fix command to resolve it.

Issue 1 — Port Already In Use

Symptom

Controller fails to start with 'address already in use: 0.0.0.0:8001' or Vite fails with 'Port 3001 is in use'.

Diagnosis

```
lsof -i :8001 # shows which process holds port 8001
lsof -i :3001 # shows which process holds port 3001
```

Fix

```
fuser -k 8001/tcp # kills process on port 8001
fuser -k 3001/tcp # kills process on port 3001
fuser -k 5000/tcp # kills GRU service if needed
```

Issue 2 — InfluxDB Not Reachable

Symptom

Data pusher logs 'Connection refused' to localhost:8086. GUI shows 0 UEs and all cells at 0% load.

Diagnosis

```
docker ps | grep influx # check if influxdb container is running
curl http://localhost:8086/ping # should return HTTP 204
```

Fix

```
cd /path/to/GUI && docker compose up -d influxdb
# Wait 5 seconds, then retry curl
```

Issue 3 — NS-3 Won't Connect to FlexRIC

Symptom

ns-3 starts and immediately prints 'E2 Connection Refused' or hangs without printing 'E2 Setup Request Sent'. FlexRIC log shows no incoming connections.

Diagnosis

```
ss -tnlp | grep 36421 # FlexRIC must show LISTEN on this port
cat /tmp/flexric.log | tail -20 # check for startup errors
```

Fix

```
pkill -f nearRT-RIC # kill any stale instance
```

```
# Restart FlexRIC, wait 5 seconds, then restart ns-3
```

Issue 4 — GRU Service Not Available

Symptom

xApp log shows 'requests.exceptions.ConnectionError' to localhost:5000. Handovers are not triggered despite overloaded cells.

Diagnosis

```
curl http://localhost:5000/health # should return {status: ok}
cat /tmp/farouk_gru.log | tail -20 # check for Python import errors
```

Fix

```
cd /path/to/5g-gui-v2 && python3 gru_xapp.py &
# Wait 3 seconds, then retry curl localhost:5000/health
```

Issue 5 — Vite Shows Blank Page

Symptom

Browser at http://localhost:3001 shows a blank white page or the canvas does not render any towers.

Diagnosis

Open the browser developer console (F12) and check for JavaScript errors. Common causes: syntax error in main.js or scene.js, missing npm packages, proxy configuration error.

```
# Check if Vite is running and listening
lsof -i :3001
# Check npm packages are installed
ls /path/to/5g-gui-v2/node_modules | head -5
```

Fix

```
cd /path/to/5g-gui-v2 && npm install # reinstall packages
npm run dev # restart Vite
```

Issue 6 — Plots Not Generated

Symptom

Simulation completed but the plots/ subdirectory is empty or missing in the sim directory.

Diagnosis

```
ls /path/to/3D_GUI_Sim_Results/sim010*/plots/
cat /path/to/sim010*/handover.csv | wc -l # check if data exists
```

Fix

```
python3 /home/omar_farouk/open-ran-clean/5g-gui-v2/generate_plots.py
# Auto-detects most recent sim dir and regenerates all plots
```

Issue 7 — No Handover Events in Results

Symptom

Simulation ran to completion but handover.csv has only the header row and decision_summary.json shows 0 total handovers.

Diagnosis

```
cat /tmp/farouk_xapp.log | tail -30 # check xApp started and subscribed
cat /tmp/flexric.log | grep 'E2 Setup' | wc -l # count E2 connections
```

If xApp log is empty, the xApp never started. If FlexRIC log shows fewer E2 Setup messages than n_mmwave, some gNBs failed to connect.

Fix

```
# Ensure xApp is started AFTER all gNBs connect to FlexRIC
# Check the E2 connection wait loop in _launch_all_task step 4
```

Issue 8 — SQLite Has Old or Duplicate Data

Symptom

GET /ctrl/decisions shows decisions from a test run that should have been discarded, or the same sim appears multiple times.

Diagnosis

```
sqlite3 /home/omar_farouk/open-ran-clean/sim_decisions.db
sqlite> SELECT sim, COUNT(*) FROM decisions GROUP BY sim;
```

Fix

```
sqlite> DELETE FROM decisions WHERE sim='sim001';
sqlite> -- or to clear everything:
sqlite> DELETE FROM decisions;
```

Issue 9 — Docker Containers Restarting

Symptom

'docker ps' shows the influxdb or gui container as 'Restarting (1) 2 seconds ago'.

Diagnosis

```
docker logs influxdb --tail 30 # check for error messages
docker logs $(docker ps -q -f name=gui) --tail 30
```

Common cause: configuration.env is missing a required variable, or the Python requirements are incompatible with the installed Python version.

Fix

```
cd GUI/ && docker compose down && docker compose up -d
# If still failing: docker compose build --no-cache gui
```

Issue 10 — Frontend Shows 0 UEs

Symptom

The 3D scene shows cell towers but no UE spheres, even though the simulation is running.

Diagnosis

```
curl http://localhost:8000/refresh-data | python3 -m json.tool | grep ues
```

If ues is an empty list: the data pusher is not running or ns-3 has not written ue_position.txt yet.

```
cat /tmp/farouk_pusher.log | tail -10 # check pusher status
```

Fix

```
# Start the pusher if not running:
cd NS3_DIR && python3 sim_data_pusher.py &
# Wait 5 seconds then check again
```

14.2 Diagnostic Commands Quick Reference

The following commands are the most useful for rapid diagnosis of any system state issue. Memorise or bookmark these.

Command	What It Shows
docker ps	Running containers and their port bindings
ss -tnlp | grep -E '36421|8086|8000|8001|3001|5000'	Which ports are currently bound and by which process
pgrep -fa 'nearRT|ns3|pusher|gru_xapp|xapp_hand'	All simulation-related processes currently running
curl -s http://localhost:8001/ctrl/status | python3 -m json.tool	Five-component health check in human-readable JSON
curl -s http://localhost:8086/ping	InfluxDB HTTP API health check (returns HTTP 204 if healthy)
curl -s http://localhost:5000/health	GRU Flask service health check
tail -f /tmp/flexric.log	Live FlexRIC log — shows E2 Setup messages, KPM subscriptions, xApp connections
tail -f /tmp/farouk_ns3.log	NS-3 simulation output — shows E2 connection attempts, simulation start messages
tail -f /tmp/farouk_xapp.log	xApp handover log — shows GRU predictions and RC command sends
tail -f /tmp/farouk_pusher.log	Data pusher log — shows InfluxDB write counts and any write errors
ls -lt 3D_GUI_Sim_Results/ | head -5	Most recent simulation result directories
sqlite3 sim_decisions.db 'SELECT sim, COUNT(*) FROM decisions GROUP BY sim'	All simulation runs stored in SQLite with handover counts

14.3 Reading FlexRIC Logs

The FlexRIC log at /tmp/flexric.log is the most informative log for diagnosing E2 connectivity issues. Key messages to look for:

- **'E2 Setup Request received'** — A gNB (ns-3 mmWave cell) has connected to FlexRIC. You should see this message 7 times for a 7-cell run. If you see fewer, some gNBs failed to connect.
- **'KPM subscription added'** — The xApp has successfully subscribed to KPM reports. This message should appear once per connected gNB after the xApp starts.
- **'RC Control message sent'** — FlexRIC has forwarded an RC handover command from the xApp to a gNB. Each such message should result in a row in handover.csv.
- **'SCTP connection closed'** — A gNB disconnected from FlexRIC. This happens when ns-3 terminates at the end of the simulation. It is expected and not an error.

Important: If /tmp/flexric.log is empty after FlexRIC has been running for 10+ seconds, FlexRIC is writing to a different location. Check that the nearRT-RIC binary was launched without output redirection overriding its default log path.

CHAPTER 15

Environment Setup — Install from Scratch

15.1 System Requirements

Requirement	Minimum	Recommended	Notes
OS	Ubuntu 20.04	Ubuntu 22.04	Tested only on Linux. macOS may work with minor changes; Windows is not supported due to SCTP and process group handling.
RAM	8 GB	16 GB	FlexRIC + ns-3 + Docker services together use ~3-4 GB at peak.
CPU	4 cores	8 cores	ns-3 is single-threaded but the build requires 4+ cores to be tolerable.
Disk	20 GB	40 GB	ns-3 build artifacts alone are ~8 GB. Sim results accumulate over time.
Python	3.8	3.10	3.8 minimum for dataclasses and typing.
Node.js	16	18	Vite 4+ requires Node 16+.
Docker	20.10	24.x	Docker Compose v2 (integrated 'docker compose') required — not the old 'docker-compose' v1.

15.2 Python Dependencies

```
pip3 install fastapi uvicorn influxdb requests flask torch numpy matplotlib pandas python-multipart
```

Key notes: 'influxdb' (not 'influxdb-client') is the v1 API client. 'torch' is PyTorch, required by the GRU inference service. 'flask' is used by gru_xapp.py for the inference server.

15.3 Node.js Dependencies (3D Frontend)

```
cd /home/omar_farouk/open-ran-clean/5g-gui-v2
npm install
```

This installs three.js, vite, and @vitejs/plugin-react (if used). The package.json pins compatible versions. After install, 'npm run dev' should start Vite on port 3001 within 5 seconds.

15.4 Docker Setup

```
# Install Docker Engine
sudo apt-get update && sudo apt-get install -y docker.io
sudo systemctl enable docker && sudo systemctl start docker
sudo usermod -aG docker $USER # allow docker without sudo
```

```
newgrp docker # activate group change

# Install Docker Compose v2 (integrated plugin)
sudo apt-get install -y docker-compose-plugin
docker compose version # should show v2.x.x
```

15.5 FlexRIC Build

FlexRIC requires CMake, a C99 compiler, and SCTP headers. The build process is documented in full in MANUAL_COMMANDS.txt. The key steps are:

```
sudo apt-get install -y libsctp-dev cmake build-essential
cd /path/to/flexric
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
```

Note: *See MANUAL_COMMANDS.txt at the project root for the exact FlexRIC repository URL, branch, and any patches needed for the mmWave E2 agent compatibility.*

15.6 NS-3 Build

NS-3 is built with its own Python-based build system (ns3). The build requires the FlexRIC E2 agent libraries to be already compiled. Full build instructions are in MANUAL_COMMANDS.txt. Key steps:

```
cd /path/to/ns3-mmwave-oran
./ns3 configure --enable-examples --enable-tests
./ns3 build scratch/gru_scenario # build only the needed scenario
```

Note: *The full ns-3 build ('./ns3 build') takes 20-40 minutes. Building only the specific scenario ('./ns3 build scratch/gru_scenario') takes 2-5 minutes.*

15.7 ML Dependencies (GRU Service)

```
pip3 install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu
```

CPU-only PyTorch is sufficient for inference with the GRU model. The model weights file (gru_model.pth or similar) must be in the same directory as gru_xapp.py. If the weights file is missing, the Flask service will fail to start.

15.8 Quick End-to-End Test

After completing all installation steps, verify the full system works:

1. Run 'bash gru.sh' from the project root. Watch for any error messages.
2. Open http://localhost:3001 in a browser. Cell towers should render immediately.
3. Wait 30-60 seconds for Docker to start. The system control panel dots should turn green one by one.
4. After ~90 seconds total (15s Docker + 15s FlexRIC + 60s sim), UE spheres should appear and animate.
5. After the simulation ends (60 seconds default), check 3D_GUI_Sim_Results/ for a new sim directory with plots.

15.9 Verifying Each Component Post-Install

After installation and before running a full simulation, verify each component individually. This narrows the search space if something fails during gru.sh.

Verify Docker and InfluxDB

```
cd GUI/ && docker compose up -d influxdb
curl http://localhost:8086/ping # expect: HTTP 204 (empty body)
docker exec -it $(docker ps -q -f name=influxdb) influx -execute 'SHOW DATABASES'
```

Verify 2D Backend

```
cd GUI/ && docker compose up -d gui
curl http://localhost:8000/health # expect: {status: ok}
```

Verify FlexRIC

```
cd NS3_DIR && ./build/examples/ric/nearRT-RIC -c flexric.conf &
sleep 3 && ss -tnlp | grep 36421 # expect: LISTEN on port 36421
pkill -f nearRT-RIC # clean up after test
```

Verify NS-3 Build

```
cd NS3_DIR && ./ns3 build scratch/gru_scenario 2>&1 | tail -5
# Expect: 'Build finished successfully' or similar success message
```

Verify GRU Service

```
cd 5g-gui-v2/ && python3 gru_xapp.py &
sleep 3 && curl http://localhost:5000/health # expect: {status: ok}
kill %1 # stop the background job
```

Verify 3D Frontend

```
cd 5g-gui-v2/ && npm run dev &
sleep 5 && curl -s http://localhost:3001/ | head -5 # expect: HTML
```

15.10 Common Post-Install Issues

The following issues commonly appear on a first-time install and their resolutions:

- **Python import errors for 'influxdb':** You installed 'influxdb-client' (v2 API) instead of 'influxdb' (v1 API). They have different module names and are incompatible. Fix: 'pip3 uninstall influxdb-client && pip3 install influxdb'.
- **'npm run dev' fails with 'vite: command not found':** node_modules was not installed. Run 'npm install' from the 5g-gui-v2/ directory first.
- **FlexRIC fails with 'error while loading shared libraries: libflexric_e2.so':** The FlexRIC shared library is not on the LD_LIBRARY_PATH. Add the FlexRIC build/lib directory: 'export LD_LIBRARY_PATH=/path/to/flexric/build/lib:$LD_LIBRARY_PATH'.
- **NS-3 build fails with 'fatal error: e2ap.h: No such file or directory':** The FlexRIC E2 agent headers are not in the ns-3 include path. Ensure the FlexRIC repository is checked out at the expected relative path and that the CMake configuration has been run with the FlexRIC path set correctly.

- **Docker 'permission denied' on docker.sock:** The current user is not in the docker group. Run 'sudo usermod -aG docker $USER && newgrp docker'.

CHAPTER 16

Results Format Reference — Every Output File

This chapter documents every file produced by a completed simulation run, with exact column definitions, example rows, and notes on how the file is written and why certain design choices were made.

16.1 handover.csv

Written live by the ns-3 C++ simulation during execution. Each row is appended to the file immediately when ns-3 processes an RC control message from the xApp and attempts the handover. The file is reset to an empty header before each new run to prevent row accumulation across runs.

Column	Type	Example	Description
time_sec	float	23.450	Simulation clock time in seconds when the handover was processed. Not wall-clock time.
ue_id	int	12	UE index, 1-based. Matches the UE index in ue_position.txt.
from_cell	int	3	Cell ID of the UE's serving cell before the handover. 0 = LTE macro cell.
to_cell	int	7	Cell ID of the target cell. Range 1-7 for mmWave cells, 0 for LTE macro.
event	str	HO_EXECUTED	String tag describing the handover event type. Always 'HO_EXECUTED' in current ns-3 code.
executed_ok	int	1	1 if ns-3 successfully completed the handover procedure; 0 if it was rejected (e.g. UE already in HO).

Example row: 23.450, 12, 3, 7, HO_EXECUTED, 1

> ***Note:*** *Only rows with executed_ok=1 are used for ping-pong detection and accuracy calculation. Rows with executed_ok=0 represent rejected handover attempts and are excluded from all statistics.*

16.2 decision_log.csv

Generated by generate_plots.py (or save_sim_results()) from handover.csv. It contains only executed_ok=1 rows, enriched with analysis metadata. Every row that is the first leg of a ping-pong pair has is_correct=False (0).

Column	Type	Example	Description
uuid	str	a3f2-...	UUID4 generated at save time. Primary key in SQLite decisions table.
sim	str	sim010	First 6 characters of the sim directory name.
time_sec	float	23.450	Copied from handover.csv.

Column	Type	Example	Description
ue_id	int	12	Copied from handover.csv.
from_cell	int	3	Copied from handover.csv.
to_cell	int	7	Copied from handover.csv.
is_pingpong	int	0	1 if this handover is the first leg of a ping-pong event (A→B followed by B→A within 5s). 0 otherwise (the handover is considered correct). Note: the column is named 'is_pingpong' in the CSV but 'is_correct' in SQLite (where 1=correct, 0=pingpong).
saved_at	str	2026-05-05T21:02:59	ISO datetime when save_sim_results() ran.

16.3 decision_summary.json

Full example output for a completed run:

```
{ "sim": "sim010", "tag": "gru_scenario", "timestamp": "2026-05-05T21:02:59.123456",
"total_handovers": 147, "pingpong_events": 8, "pingpong_rate_pct": 5.44,
"correct_decisions": 139, "total_decisions": 147, "accuracy_pct": 94.56 }
```

accuracy_pct is computed as: (total_handovers - pingpong_events) / total_handovers * 100. A ping-pong event counts as one penalised decision (the outgoing A→B leg), not two. So 8 ping-pong events in 147 handovers gives accuracy = 139/147 * 100 = 94.56%.

```
accuracy_pct = (total_handovers - pingpong_events) / total_handovers * 100
```

16.4 summary.txt

Plain-text version of decision_summary.json, written for human readability:

```
Simulation: sim010 (gru_scenario) Saved at: 2026-05-05T21:02:59.123456 Total
handovers executed: 147 Ping-pong events: 8 Ping-pong rate: 5.44% Correct decisions:
139 Total decisions: 147 GRU Accuracy: 94.56%
```

16.5 Plots Reference

Filename	Type	X-Axis	Y-Axis	What to Look For
decision_quality.png	Scatter plot	Simulation time (seconds)	0 = Ping-pong / 1 = Correct	Red X clusters in time indicate periods when GRU made bursty poor decisions — look for correlation with high cell load
handovers_over_time.png	Cumulative line	Simulation time (seconds)	Cumulative handover count	Flat regions = no handovers (all UEs well-served); steep slope = burst of handovers (possible instability)

Filename	Type	X-Axis	Y-Axis	What to Look For
ho_per_ue.png	Bar chart	UE ID (numeric)	Total handover count	Spikes on specific UEs indicate high-mobility users or UEs at cell edges; even distribution is ideal
ho_activity.png	Histogram	Simulation time bin (5-second windows)	Handovers in window	Early-sim spikes are normal (initial association); mid-sim spikes may indicate load imbalance responses

16.6 Complete Output Directory Layout

After a successful simulation run with save_sim_results(), the output directory has the following exact structure:

```
3D_GUI_Sim_Results/ sim010_20260505_210259_gru_scenario/ handover.csv # raw NS-3 handover events lstm_features.csv # feature vectors sent to GRU (if run) decision_log.csv # enriched handover events with PP flags decision_summary.json # aggregate statistics summary.txt # human-readable statistics flexric.log # copy of /tmp/flexric.log at run end farouk_ns3.log # copy of /tmp/farouk_ns3.log farouk_xapp.log # copy of /tmp/farouk_xapp.log farouk_pusher.log # copy of /tmp/farouk_pusher.log farouk_gru.log # copy of /tmp/farouk_gru.log plots/ decision_quality.png handovers_over_time.png ho_per_ue.png ho_activity.png
```

Note: *Not all log files will be present in every run. If a component was not started (e.g. GRU service not used), its log copy will be skipped. The directory naming uses zero-padded three-digit sim numbers (sim001 through sim999). After sim999, _next_sim_number() will produce sim1000 (four digits) — this is not handled gracefully by all display code.*

16.7 File Lifecycle — When Each File Is Written and Read

File	Written By	When Written	Read By	When Read
handover.csv	ns-3 C++	Each time an RC handover command is executed during sim	generate_plots.py, save_sim_results()	At end of simulation
lstm_features.csv	xapp_handover_gru (C)	Each time the xApp queries the GRU service	save_sim_results()	At end of simulation (copy only)
ue_position.txt	ns-3 C++	Every 0.05 sim-seconds	sim_data_pusher.py	Every 3 seconds (read + clear)
du-cell-N.txt	ns-3 C++	Every 0.05 sim-seconds	sim_data_pusher.py	Every 3 seconds (read + clear)
decision_log.csv	generate_plots.py	Once, after simulation ends	3D frontend /ctrl/decisions	On demand via API

File	Written By	When Written	Read By	When Read
decision_summary.json	generate_plots.py	Once, after simulation ends	3D frontend /ctrl/last-result	On demand via API
sim_decisions.db (SQLite)	_write_decisions_to_db()	Once per sim, at save time	/ctrl/decisions endpoint	On every API request

CHAPTER 17

Database Query Examples — API and DB

This chapter provides ready-to-run commands for querying the SQLite decisions database and the InfluxDB time-series database, plus Python one-liners for programmatic access.

17.1 GET /ctrl/decisions — curl Examples

```
# All decisions (up to default limit of 500)
curl http://localhost:8001/ctrl/decisions | python3 -m json.tool
```

```
# Only decisions from sim007
curl 'http://localhost:8001/ctrl/decisions?sim=sim007'
```

```
# Last 50 decisions across all sims
curl 'http://localhost:8001/ctrl/decisions?limit=50'
```

```
# Filter and count ping-pong events via jq
curl -s 'http://localhost:8001/ctrl/decisions?sim=sim010' | python3 -c "import
json,sys; d=json.load(sys.stdin)['decisions']; print('PP events:', sum(1 for r in d
if not r['is_correct']))"
```

17.2 Python One-Liners for Analysis

Count ping-pong events in the last completed sim:

```
python3 -c " import sqlite3, collections con =
sqlite3.connect('/home/omar_farouk/open-ran-clean/sim_decisions.db') rows =
con.execute('SELECT sim, is_correct FROM decisions').fetchall() from collections
import Counter sims = Counter(r[0] for r in rows) pp = Counter(r[0] for r in rows if
not r[1]) for s in sorted(sims): print(f'{s}: {sims[s]} total, {pp[s]} PP') "
```

Compute accuracy percentage per sim from SQLite:

```
python3 -c " import sqlite3 con =
sqlite3.connect('/home/omar_farouk/open-ran-clean/sim_decisions.db') rows =
con.execute( 'SELECT sim, COUNT(*) as tot, SUM(CASE WHEN is_correct=0 THEN 1 ELSE 0
END) as pp ' 'FROM decisions GROUP BY sim ORDER BY sim' ).fetchall() for sim, tot, pp
in rows: acc = (tot-pp)/tot*100 if tot else 0 print(f'{sim}: {tot} HOs, {pp} PP,
accuracy={acc:.1f}%') "
```

17.3 SQLite Direct Queries

```
sqlite3 /home/omar_farouk/open-ran-clean/sim_decisions.db
```

View all sims with handover counts:

```
SELECT sim, COUNT(*) as total_hos, SUM(CASE WHEN is_correct=0 THEN 1 ELSE 0 END) as
pp_count FROM decisions GROUP BY sim ORDER BY sim;
```

Expected output: one row per sim label showing total handovers and ping-pong count.

Accuracy per sim:

```
SELECT sim, ROUND(100.0 * SUM(is_correct) / COUNT(*), 2) AS accuracy_pct FROM
decisions GROUP BY sim ORDER BY sim;
```

All ping-pong events for sim010:

```
SELECT time_sec, ue_id, from_cell, to_cell, saved_at FROM decisions WHERE
sim='sim010' AND is_correct=0 ORDER BY time_sec;
```

Clear all data for a specific sim (irreversible):

```
DELETE FROM decisions WHERE sim='sim001';
```

Count total decisions across all sims:

```
SELECT COUNT(*) FROM decisions;
```

17.4 InfluxDB Queries via Docker Exec

```
# Open InfluxDB CLI docker exec -it $(docker ps -q -f name=influxdb) influx
```

```
> USE influx
```

Show all measurements currently in the database:

```
> SHOW MEASUREMENTS
```

Get the last 5 SINR values for UE 5:

```
> SELECT * FROM "ue_5_l3 serving sinr" LIMIT 5
```

Get PRB usage history for cell 3:

```
> SELECT * FROM "du-cell-3_dlprbusage" LIMIT 10
```

Get UE 12 position history:

```
> SELECT * FROM "ue_position_x_12" LIMIT 5
> SELECT * FROM "ue_position_y_12" LIMIT 5
```

Clear all time-series data (keeps database structure):

```
> DROP SERIES FROM /.*/
```

Drop and recreate database (full reset):

```
> DROP DATABASE influx
> CREATE DATABASE influx
> USE influx
```

17.5 HTTP API to InfluxDB (from host)

InfluxDB also accepts queries via HTTP. These can be run from the host terminal without entering the container:

```
# Query via HTTP API curl -G 'http://localhost:8086/query' \ --data-urlencode
'db=influx' \ --data-urlencode 'q=SELECT * FROM "ue_5_l3 serving sinr" LIMIT 5'
```

```
# Write a test point via line protocol curl -X POST
'http://localhost:8086/write?db=influx' \ --data-binary 'test_measurement
value=42.0'
# Drop and recreate database curl -X POST 'http://localhost:8086/query' \
--data-urlencode 'q=DROP DATABASE influx' curl -X POST 'http://localhost:8086/query'
\ --data-urlencode 'q=CREATE DATABASE influx'
```

17.6 Multi-Sim Analysis Scripts

The following Python scripts perform analysis across multiple simulation runs. They can be run directly from the terminal against the SQLite database.

Compare accuracy across all runs

```
#!/usr/bin/env python3 import sqlite3 DB =
'/home/omar_farouk/open-ran-clean/sim_decisions.db' con = sqlite3.connect(DB) rows =
con.execute( 'SELECT sim, COUNT(*) tot, ' 'SUM(CASE WHEN is_correct=0 THEN 1 ELSE 0
END) pp ' 'FROM decisions GROUP BY sim ORDER BY sim' ).fetchall() print(f"{'Sim':<10}
{'Handovers':>10} {'PP Events':>10} {'Accuracy':>10}") print('-' * 44) for sim, tot,
pp in rows: acc = (tot - pp) / tot * 100 if tot else 0 print(f"{sim:<10} {tot:>10}
{pp:>10} {acc:>9.1f}%")
```

Find the UE with the most ping-pong events across all runs

```
#!/usr/bin/env python3 import sqlite3, collections DB =
'/home/omar_farouk/open-ran-clean/sim_decisions.db' con = sqlite3.connect(DB) rows =
con.execute( 'SELECT ue_id, COUNT(*) FROM decisions ' 'WHERE is_correct=0 GROUP BY
ue_id ORDER BY COUNT(*) DESC LIMIT 10' ).fetchall() print('Top 10 UEs by ping-pong
event count:') for ue_id, count in rows: print(f' UE {ue_id}: {count} ping-pong
events')
```

Find most frequent source-destination cell pairs for ping-pong

```
#!/usr/bin/env python3 import sqlite3 DB =
'/home/omar_farouk/open-ran-clean/sim_decisions.db' con = sqlite3.connect(DB) rows =
con.execute( 'SELECT from_cell, to_cell, COUNT(*) cnt ' 'FROM decisions WHERE
is_correct=0 ' 'GROUP BY from_cell, to_cell ORDER BY cnt DESC LIMIT 10' ).fetchall()
print('Most frequent ping-pong cell pairs:') for fc, tc, cnt in rows: print(f' Cell
{fc} -> Cell {tc}: {cnt} ping-pong events')
```

17.7 InfluxDB Query Patterns for Custom Dashboards

The following InfluxQL patterns are useful when building custom Grafana panels or Python analysis scripts that query InfluxDB directly.

Average SINR across all UEs at a point in time

```
SELECT mean(value) FROM /ue_.*_l3 serving sinr/ WHERE time > now() - 10s GROUP BY
time(5s)
```

Cell load time series for cell 3

```
SELECT value FROM "du-cell-3_dlprbusage" WHERE time > now() - 5m ORDER BY time ASC
```

UE 5 position history

```
SELECT value FROM "ue_position_x_5", "ue_position_y_5" WHERE time > now() - 2m ORDER
BY time ASC
```

Count measurements written per minute (push health check)

```
SELECT count(value) FROM /.*/ WHERE time > now() - 5m GROUP BY time(1m)
```

17.8 Exporting Decisions to CSV for External Analysis

The SQLite decisions database can be exported to CSV for analysis in Excel, pandas, R, or any other tool.

```
# Export all decisions to CSV sqlite3 -csv -header
/home/omar_farouk/open-ran-clean/sim_decisions.db \ 'SELECT * FROM decisions ORDER BY
sim, time_sec' \ > /tmp/all_decisions.csv

# Export only sim010 decisions sqlite3 -csv -header
/home/omar_farouk/open-ran-clean/sim_decisions.db \ "SELECT * FROM decisions WHERE
sim='sim010' ORDER BY time_sec" \ > /tmp/sim010_decisions.csv

# Load in pandas for analysis import pandas as pd df =
pd.read_csv('/tmp/all_decisions.csv')
print(df.groupby('sim')['is_correct'].agg(['sum', 'count']))
```

17.9 Reset and Housekeeping Commands

Use these commands to clean up between experiment runs:

```
# Clear all InfluxDB data but keep database structure

curl -X POST 'http://localhost:8086/query' --data-urlencode 'q=DROP SERIES FROM /.*/'
-d 'db=influx'

# Archive old sim results (move to backup)

mv /home/omar_farouk/open-ran-clean/3D_GUI_Sim_Results
/home/omar_farouk/sim_backup_$(date +%Y%m%d)

mkdir /home/omar_farouk/open-ran-clean/3D_GUI_Sim_Results

# Back up the decisions database

cp sim_decisions.db sim_decisions_backup_$(date +%Y%m%d).db

# Clear all sim data from SQLite (destructive, irreversible)

sqlite3 sim_decisions.db 'DELETE FROM decisions;'
```

Important: There is no automatic backup or rotation of 3D_GUI_Sim_Results/. As each simulation run produces 5-15 MB of logs and images, after 100 runs the directory will be 500 MB+. Archive old results periodically, especially before production experiments.

CHAPTER 18

GRU Machine Learning Model — Architecture and Training

18.1 Why a GRU for Handover Decisions?

A handover decision is fundamentally a sequence prediction problem. The decision of whether to hand UE X from cell A to cell B should not be based solely on the current SINR snapshot — it should account for the recent trajectory of SINR values. A UE whose SINR on cell B has been rising for the past 300 milliseconds is a much stronger handover candidate than a UE whose SINR on cell B momentarily spiked but is otherwise low.

Traditional handover algorithms (A3 event-based, TTT hysteresis) handle this with fixed time-to-trigger timers that require manual tuning. A Gated Recurrent Unit (GRU) neural network can learn this temporal pattern from data, adapting to the specific propagation environment and mobility patterns of the simulated network without manual parameter tuning.

GRU was chosen over LSTM (Long Short-Term Memory) for three reasons: GRU has fewer parameters (2 gates vs 3), which means faster training and inference; GRU achieves comparable accuracy to LSTM on the short-horizon sequences used here (window of 10 SINR samples = 500ms of simulation time); and the simpler weight structure makes the model easier to inspect and debug.

Key Point: The GRU model is a sequence classifier, not a sequence generator. It takes a fixed-length window of recent SINR measurements as input and outputs a single target cell index as its prediction. The output is a probability distribution over all cells; the argmax is taken as the final decision.

18.2 GRU Architecture

The model architecture used in this project is a two-layer GRU followed by a fully connected output layer. All layer sizes are configurable via constants at the top of gru_xapp.py.

Layer	Type	Input Size	Output Size	Notes
Input	—	window_size × n_features	—	Each timestep has n_features = (n_cells * 2) features: serving SINR + neighbor SINR per cell
GRU Layer 1	GRU	n_features	hidden_size (64)	Returns full sequence output; dropout=0.2 applied
GRU Layer 2	GRU	64	hidden_size (64)	Returns only final hidden state (not sequence)
FC Layer	Linear	64	n_cells (8)	Maps hidden state to cell count logits
Output	Softmax	n_cells	n_cells	Probability distribution over target cells

18.3 Input Feature Vector Construction

For each handover decision, the xApp (xapp_handover_gru.c) constructs a feature vector from the most recent KPM report. The feature vector has the following structure:

- **Serving cell SINR (1 value):** The SINR of the UE's current serving cell in dB, as reported in the E2 KPM indication message's L3 Serving SINR field.
- **Neighbor cell SINRs (N values, one per mmWave cell):** The SINR measured for each of the N mmWave neighbor cells, ordered by cell index. Missing values (cells not in range) are filled with -140 dBm (the minimum measurable SINR).
- **Current load per cell (N values):** The most recently reported DL PRB usage for each cell, as a float in [0, 1]. This allows the model to incorporate load balancing objectives into its decisions.

The xApp maintains a rolling window of the last 10 feature vectors in a circular buffer. When the window is full, it serialises the buffer to JSON and sends a POST request to the GRU Flask service. The service runs the model and returns the predicted target cell index.

```
# Feature vector structure (Python representation): { 'features': [ # timestep 0
(oldest): [serving_sinr, neigh_sinr_1, ..., neigh_sinr_7, load_0, ..., load_7], #
timestep 1: [...], # ... # timestep 9 (most recent): [...] ] }
```

18.4 Training Data Generation

The GRU model is trained on simulation data generated by running ns-3 with a reference handover algorithm (the standard A3 event trigger with TTT=320ms and hysteresis=3dB). For each handover event in the training data:

1. **Extract the window:** The 10 feature vectors immediately preceding the handover time are extracted from lstm_features.csv.
2. **Label the sample:** The target cell of the handover (to_cell field from handover.csv) is the ground-truth label for this window.
3. **Filter ping-pong events:** Samples where the handover was subsequently identified as a ping-pong event are excluded from training, since they represent incorrect reference decisions.
4. **Balance classes:** Because cells at cell edges attract more handovers than central cells, the training set is class-balanced using sklearn.utils.resample to prevent the model from biasing toward high-frequency cells.

18.5 Training Procedure

Training is performed in a Jupyter notebook (or standalone Python script). Key hyperparameters:

Hyperparameter	Value	Rationale
Sequence length (window)	10 timesteps	Each timestep is 50ms (0.05s), so 10 = 500ms look-back — slightly longer than TTT=320ms
Hidden size	64	Sufficient capacity for 8-cell topology; larger sizes did not improve validation accuracy

Hyperparameter	Value	Rationale
Batch size	64	Fits in GPU memory even with large datasets; smaller batches were slower without accuracy benefit
Learning rate	0.001	Adam optimizer default; reduces automatically on plateau via ReduceLROnPlateau
Epochs	50 (early stopping)	Validation loss typically plateaus by epoch 30-40
Dropout rate	0.2	Applied between GRU layers to reduce overfitting on small datasets
Loss function	CrossEntropyLoss	Appropriate for multi-class classification over 8 cell targets
Optimizer	Adam (lr=0.001)	Adam with default betas; no weight decay

18.6 Inference Service — gru_xapp.py

gru_xapp.py is a Flask server that wraps the trained GRU model. It exposes two endpoints: GET /health (returns {status: ok, model: gru}) and POST /predict (accepts the feature window, returns the predicted target cell).

On startup, gru_xapp.py loads the model weights from a .pth file using torch.load(). The model is set to eval() mode (disables dropout). Inference runs on CPU — GPU is not required for a single forward pass on a 10x16 input tensor. Typical inference latency is < 2ms on modern hardware.

```
# Inference path in gru_xapp.py: @app.route('/predict', methods=['POST']) def predict(): data = request.get_json() x = torch.tensor(data['features'], dtype=torch.float32) x = x.unsqueeze(0) # add batch dimension: (1, seq_len, n_features) with torch.no_grad(): output = model(x) # shape: (1, n_cells) probs = torch.softmax(output, dim=1) pred = torch.argmax(probs, dim=1).item() return jsonify({'target_cell': pred, 'confidence': probs[0][pred].item()})
```

18.7 Interpreting GRU Accuracy Metrics

The decision_summary.json reports 'accuracy_pct' which is defined as the percentage of executed handovers that were NOT identified as ping-pong events. This is a conservative accuracy metric — it penalises the GRU only for decisions that provably caused oscillation (A→B followed by B→A within 5 seconds). A handover that moved a UE to a slightly suboptimal cell but did not cause oscillation is counted as correct.

A GRU accuracy of 90%+ in the simulation environment is considered good performance. The reference A3 algorithm with TTT=320ms and hysteresis=3dB typically achieves 85-90% accuracy on the same topology with the same mobility model, which gives the GRU a meaningful but not overwhelming advantage.

GRU Accuracy = (Total Handovers - Ping-Pong Events) / Total Handovers * 100

Ping-Pong Rate = Ping-Pong Events / Total Handovers * 100

Note: *These metrics measure handover decision quality in aggregate across the simulation. They do NOT measure per-UE quality or per-cell quality. Use the ho_per_ue.png plot and the SQLite GROUP BY queries to drill down to specific UEs or cell pairs.*

CHAPTER 19

O-RAN Architecture Context

This chapter provides the O-RAN standards context that explains why the system is designed the way it is. Understanding the O-RAN architecture is essential for understanding why FlexRIC, the E2 interface, xApps, KPM reports, and RC control messages exist as separate concepts.

19.1 What Is O-RAN?

O-RAN (Open Radio Access Network) is an industry alliance specification that disaggregates the traditional monolithic base station into separate functional units connected by open, standardised interfaces. The key insight is that a radio base station can be decomposed into a Radio Unit (RU), a Distributed Unit (DU), and a Central Unit (CU), each potentially running on standard commercial hardware. Control-plane intelligence is moved to a separate RAN Intelligent Controller (RIC) that communicates with the base stations via the E2 interface.

The motivation for O-RAN is to break vendor lock-in. In a traditional 4G/5G network, the RAN hardware and software are a proprietary stack from a single vendor (Ericsson, Nokia, Huawei, etc.). O-RAN allows operators to mix hardware from different vendors and to deploy custom control logic via xApps running on the RIC, rather than relying on the vendor's embedded algorithms.

19.2 The RIC Hierarchy

O-RAN defines two types of RIC with different latency targets:

Non-Real-Time RIC (Non-RT RIC)

Operates on timescales of seconds to minutes. Responsible for model training, policy configuration, and A1 interface policy distribution to the nearRT-RIC. In this project, the non-RT RIC role is played by the simulation controller (controller.py) which configures the simulation parameters before each run.

Near-Real-Time RIC (nearRT-RIC)

Operates on timescales of 10ms to 1 second. Responsible for real-time RAN optimisation based on E2 reports from gNBs. Hosts xApps that subscribe to KPM reports and send RC control messages. In this project, FlexRIC is the nearRT-RIC implementation.

19.3 The E2 Interface

The E2 interface connects gNBs (ns-3 simulation) to the nearRT-RIC (FlexRIC). It uses SCTP (Stream Control Transmission Protocol) as the transport layer, which provides ordered, reliable delivery over IP. The application protocol on top of SCTP is E2AP (E2 Application Protocol), defined by O-RAN in a document called E2AP Specification.

E2AP defines three main message types used in this project:

- **E2 Setup Request / Response:** The initial handshake. When ns-3 starts, each gNB's E2 agent sends an E2 Setup Request to FlexRIC on port 36421. FlexRIC responds with an E2 Setup Response confirming the

connection. This is why the launch-all task waits for 7 ESTABLISHED SCTP connections before starting the xApp — one connection per mmWave gNB.

- **E2 Subscription Request / Response:** The xApp (via FlexRIC) sends a subscription to each gNB requesting periodic KPM reports. The subscription specifies the report period (indicationPeriodicity=0.05 seconds), the list of KPM measurements requested (SINR, PRB usage, etc.), and the E2 RAN function ID (KPM_E2functionID=2).
- **E2 Indication:** Once subscribed, each gNB sends periodic E2 Indication messages carrying the KPM report. Each Indication contains a list of UE-level measurements for all UEs currently served by that gNB.
- **E2 Control (RC):** The xApp sends an E2 Control message to a gNB requesting a handover. The message specifies the target UE (by RNTI) and the target cell (by NCI). This uses E2 RAN function ID RC_E2functionID=3.

19.4 KPM Service Model

KPM (Key Performance Measurements) is one of the O-RAN defined E2 Service Models (E2SM). An E2 Service Model defines the structure and semantics of the data exchanged via E2. KPM defines:

- **Measurement types:** SINR (L3 Serving SINR, L3 Neighbor SINR), PRB usage (DL PRB usage, UL PRB usage), throughput (DL PDCP bit rate, UL PDCP bit rate), latency (DL PDCP delay), and error rates.
- **Granularity period:** How frequently measurements are reported. In this project: 0.05 simulation seconds (50 ms). This is the value of the indicationPeriodicity command-line flag passed to ns-3.
- **Report trigger:** Periodic (every granularity period) rather than event-triggered. This gives the xApp a continuous stream of measurements rather than only on events.

19.5 RC Service Model

RC (RAN Control) is the E2 Service Model used for issuing control commands to gNBs. In this project, the xApp uses RC to request handovers. The RC control message contains:

- **UE ID:** The C-RNTI of the UE to be handed over.
- **Target cell ID:** The NCI (NR Cell Identity) of the target cell.
- **Control action ID:** Identifies this as a handover request (as opposed to other RC control actions like beam management or scheduler parameter changes).

19.6 FlexRIC as nearRT-RIC Implementation

FlexRIC is an open-source nearRT-RIC implementation developed by EURECOM. It implements the E2AP protocol stack in C and provides an xApp SDK that allows xApps to be written in C, Python, or Rust. In this project, the xApp (xapp_handover_gru) is written in C and uses FlexRIC's C SDK.

FlexRIC runs as a single process (nearRT-RIC binary) that:

1. Listens on port 36421 for incoming SCTP E2 connections from gNBs.
2. Maintains a registry of all connected E2 nodes (gNBs) and their supported RAN function IDs.
3. Receives xApp registrations and routes E2 Indication messages to subscribed xApps.
4. Forwards E2 Control messages from xApps to the appropriate gNB.

5. Logs all activity to /tmp/flexric.log.

19.7 How This Project Maps to O-RAN

The table below maps each component of this project to its O-RAN standard equivalent role.

Project Component	O-RAN Standard Role	Standard Interface
FlexRIC binary (nearRT-RIC)	Near-RT RIC	E2 (southbound to gNBs), A1 (northbound to Non-RT RIC — not used)
xapp_handover_gru (C binary)	xApp	Runs on Near-RT RIC, receives E2 KPM Indications, sends E2 RC Control
ns-3 gNBs (mmWave+LTE)	CU/DU/RU stack (gNB)	E2 (northbound to nearRT-RIC), air interface (simulated)
gru_xapp.py (Flask)	Non-standard AI inference	HTTP (internal, not an O-RAN interface)
controller.py (FastAPI)	O1 interface consumer (approximate)	HTTP REST (not standard O1, but similar orchestration role)
sim_data_pusher.py	No O-RAN equivalent	Internal — reads ns-3 CSV output, writes to InfluxDB
InfluxDB	PM (Performance Management) storage	Not an O-RAN component — proprietary storage backend
3D GUI frontend	No O-RAN equivalent	Visualisation layer — not part of the O-RAN architecture

19.8 E2 Port 36421 — Why This Number?

Port 36421 is the IANA-registered port number for the E2AP protocol over SCTP. It was assigned specifically for O-RAN E2 interface use. All standard-compliant nearRT-RIC implementations must listen on this port. FlexRIC binds to 0.0.0.0:36421 by default (all network interfaces), which means it also accepts connections from remote machines. In this project, ns-3 connects to 127.0.0.1:36421 (localhost) because both FlexRIC and ns-3 run on the same machine.

> ***Note:*** *SCTP is not TCP. It is a transport protocol that provides multiple independent streams within a single association, avoiding head-of-line blocking between different E2 message types. Linux requires the 'libsctp-dev' package and the 'sctp' kernel module to be loaded for SCTP sockets to work.*

CHAPTER 20

Glossary — Key Terms and Abbreviations

This glossary defines all technical terms, abbreviations, and system-specific names used in this guide. Terms are organised alphabetically within categories.

20.1 O-RAN and 5G Standards Terms

Term	Full Form	Definition
A3 Event	—	LTE/NR handover trigger event: UE's neighbor cell SINR exceeds serving cell SINR by a threshold (hysteresis) for longer than TTT.
CU	Central Unit	The upper part of the gNB responsible for RRC and PDCP layer processing. May be split into CU-CP (control plane) and CU-UP (user plane).
DU	Distributed Unit	The lower part of the gNB responsible for RLC, MAC, and lower physical layer. Communicates with CU via F1 interface.
E2AP	E2 Application Protocol	The application-layer protocol used on the E2 interface between gNBs and the nearRT-RIC. Defines Setup, Subscription, Indication, and Control message types.
E2SM-KPM	E2 Service Model — Key Performance Measurements	O-RAN service model defining the structure and semantics of performance measurement reports over E2.
E2SM-RC	E2 Service Model — RAN Control	O-RAN service model defining control commands (handover, beam management, scheduler) sent from xApp to gNB via E2.
gNB	Next-Generation Node B	5G base station. In ns-3 mmWave, implemented as a CU+DU stack connected to UEs via simulated mmWave channel.
hoSinrDifference	Handover SINR Difference	NS-3 command-line parameter (--hoSinrDifference): the minimum SINR gain a UE must see on the target cell vs serving cell to trigger a handover. Default 3 dB.
indicationPeriodicity	—	NS-3 command-line parameter: how often KPM reports are sent to FlexRIC in simulation seconds. Default 0.05 (= 50ms).
KPM	Key Performance Measurements	Metrics reported by gNBs to the nearRT-RIC via E2 Indication messages. In this project: SINR, PRB usage, throughput, latency, UE count.
LTE	Long Term Evolution	4G cellular standard. In this project, one LTE macro cell (eNB) provides anchor coverage for UEs between mmWave handovers.

Term	Full Form	Definition
mmWave	Millimetre Wave	High-frequency (24-100 GHz) 5G radio. High capacity but short range and sensitivity to obstacles. Used for the 7 small cell gNBs in this project.
nearRT-RIC	Near-Real-Time RAN Intelligent Controller	O-RAN RIC component operating at 10ms-1s latency. Hosts xApps. Implemented by FlexRIC in this project.
NCI	NR Cell Identity	Unique identifier for an NR (5G NR) cell. Used in E2 RC control messages to identify the target handover cell.
Non-RT RIC	Non-Real-Time RIC	O-RAN RIC component operating at >1 second latency. Handles model training and A1 policy distribution.
O-RAN	Open Radio Access Network	Industry alliance and set of specifications for disaggregated, open-interface RAN architectures.
PRB	Physical Resource Block	The smallest schedulable unit of radio spectrum in LTE/NR. One PRB = 12 subcarriers x 1 OFDM symbol. PRB usage is the fraction of available PRBs in use.
RC	RAN Control	E2 Service Model for issuing control commands to gNBs, including handover requests.
RIC	RAN Intelligent Controller	The O-RAN component that hosts xApps and communicates with gNBs via E2.
RNTI	Radio Network Temporary Identifier	Temporary identifier assigned to a UE by its serving cell. Used in E2 RC messages to identify the UE to be handed over.
SCTP	Stream Control Transmission Protocol	Transport protocol used by E2AP. Provides reliable, ordered delivery with multiple independent streams. Requires libsctp-dev on Linux.
SINR	Signal-to-Interference-plus-Noise Ratio	Key radio quality metric in dB. Higher SINR means better signal quality. The primary input to GRU handover decisions.
TTT	Time-To-Trigger	In A3 event-based handover: the duration a UE must meet the handover condition before the handover is triggered. Default in this project: 320ms.
UE	User Equipment	Mobile device. In ns-3, simulated as a mobile node following a RandomWaypoint mobility model within the simulation area.
xApp	Executable Application	A microservice running on the nearRT-RIC that receives E2 reports and sends E2 control messages. In this project: xapp_handover_gru (C binary).

20.2 System-Specific Terms

Term	Definition
3D_GUI_Sim_Results/	Directory containing numbered simulation output directories (sim001, sim002, ...).
_alive(key)	Internal controller.py function: returns True if the process stored under 'key' in _procs is still running.
_build_decision_log()	Controller.py function that produces the enriched decision list with is_correct flags.
_calc_pingpong()	Controller.py function that runs ping-pong detection and returns total/pp/rate_pct.
_launch_all_task	The async coroutine in controller.py that runs the 10-step simulation launch sequence in the background.
_popen()	Controller.py helper: kills existing process, spawns new one, stores Popen in _procs.
_procs	Module-level dict in controller.py mapping string keys to subprocess.Popen objects.
build/examples/ric/nearRT-RIC	Path to the FlexRIC binary relative to NS3_DIR.
configuration.env	Environment variable file for Docker services; sets InfluxDB and Grafana credentials.
controller.py	FastAPI orchestration backend running on port 8001; the brain of the system.
data_controller.py	FastAPI router in the 2D backend Docker container; implements /refresh-data.
decision_log.csv	Enriched handover event log with UUID, sim label, and ping-pong flags.
decision_summary.json	JSON file with aggregate accuracy statistics for a completed simulation run.
demo mode	Mode the 3D frontend enters when /api/refresh-data is unreachable; uses random walk for cell loads.
FLEXRIC_BIN	Path constant in controller.py pointing to the nearRT-RIC binary.
gru.sh	Shell script for terminal-based simulation launch. Accepts simTime, nUEs, nCells arguments.
gru_xapp.py	Flask server wrapping the trained GRU PyTorch model; exposes /predict and /health.
GUI_DIR	Path constant pointing to the docker-compose.yml directory (GUI/).
handover.csv	Live-written CSV by ns-3 C++ code; contains one row per RC-triggered handover attempt.
influx (database name)	The InfluxDB database name used throughout the project. Set by INFLUXDB_DB=influx.

Term	Definition
kill_sim.sh	Shell script for clean simulation shutdown; also calls generate_plots.py.
LOG dict	Module-level dict in controller.py mapping component names to /tmp/ log file paths.
lstm_features.csv	Feature vector log written by xapp_handover_gru; one row per GRU query.
NET	Global JavaScript object in main.js holding the current simulation state (cells, UEs, sim mode).
NS3_DIR	Path constant in controller.py pointing to the ns3-mmwave-oran build directory.
on_page() / on_first_page()	ReportLab page callback functions in this PDF generator; draw header line and footer.
pollBackend()	JavaScript function in main.js; polls /api/refresh-data every 1.5 seconds.
pollCtrlStatus()	JavaScript function in main.js; polls /ctrl/status every 3 seconds.
RESULTS_DIR	Path constant in controller.py: /home/omar_farouk/open-ran-clean/3D_GUI_Sim_Results.
save_sim_results()	Controller.py function called at end of each simulation; archives files, writes SQLite, generates plots.
sim_data_pusher.py	Python bridge: reads ns-3 CSV files every 3 seconds and writes to InfluxDB.
sim_decisions.db	SQLite database at project root; stores all handover decisions across all simulation runs.
SimParams	Pydantic model in controller.py defining the request body for /ctrl/simulation/start and /ctrl/launch-all.
vite.config.js	Vite configuration file; sets dev server port 3001 and four proxy rules to backend services.
xapp_handover_gru	The C binary xApp that runs on FlexRIC; receives KPM E2 reports and sends RC handover commands.
xApp_trigger.py	Trigger server for non-GRU xApps; listens on port 38868 for start/stop HTTP POST messages.